

《编码》逐章精读指南

目录

- 《编码》逐章精读指南
 - 目录
 - 第1章：至亲密友
 - 第2章：编码与组合
 - 第3章：布莱叶盲文与二进制码
 - 第4章：手电筒的剖析
 - 第5章：绕过拐角的通信
 - 第6章：电报机与继电器
 - 第7章：我们的十个数字
 - 第8章：十的替代品
 - 第9章：二进制数的奥秘
 - 第10章：逻辑与开关
 - 第11章：门
 - 第12章：二进制加法器
 - 第13章：如何实现减法
 - 第14章：反馈与触发器
 - 第15章：字节与十六进制
 - 第16章：存储器组织
 - 第17章：自动操作
 - 第18章：从算盘到芯片
 - 第19章：两种典型的微处理器
 - 第20章：ASCII码及其他
 - 第21章：总线
 - 第22章：操作系统
 - 第23章：定点数与浮点数
 - 第24章：高级语言与低级语言
 - 第25章：图形化革命
 - 全书的最后一页：写给你自己

第1章：至亲密友

两个小孩，一扇窗户，一支手电筒。这是计算机通信的起点。

没有机房，没有光纤，没有卫星。只有一对想聊天的邻居小孩，在父母关灯后，用手电筒的闪光来传递字母。他们在窗户之间，用最原始的光，完成了人类历史上最简单的“数字通信”。

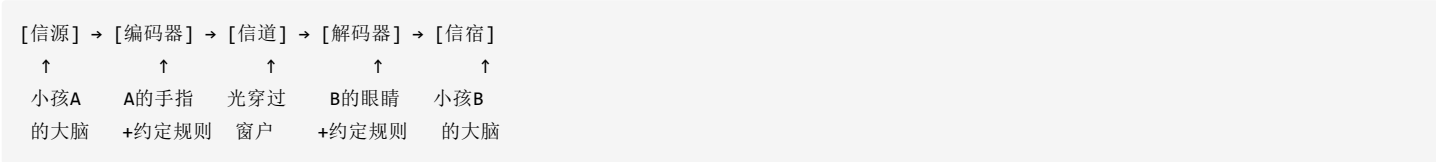
1.1 闪光即信息：比特的诞生

手电筒只有两种状态——亮，或者灭。这两种状态，就是计算机世界里最小的信息单位：**1 bit**。

bit 不关心你在传什么——情书、军情、求救信号。它只关心一件事：“是”或“否”。亮是1，灭是0。你打开电脑看到的一切——文字、图像、音乐、程序——归根结底都是数十亿个这样的“是”或“否”堆叠在一起的结果。

但光有闪光还不够。如果两个小孩没有提前约定“一个短闪 = A，两个短闪 = B”，那闪光就只是闪光，不是信息。**编码的本质，是让物理状态有了语义。**发送方和接收方共享一套规则——用规则把信息变成信号，再用规则把信号变回信息。

把这两个小孩的故事抽出来，就是一个通用的通信模型：



把这个模型和今天的网卡、光纤、Wi-Fi对比一下：只是把"手指按手电筒"换成了"网卡发送电信号"，把"光穿过窗户"换成了"信号在网线或光纤中传播"。物理介质变了，逻辑没变。

1.2 三个伏笔：本章没解决的问题

这一章留下的问题，比解决的问题更多。这是故意的——这些问题会在后续章节中——解开。

1. 怎么知道一个闪光结束了？

接收方盯着手电筒，看到亮了一下。但这个"一下"是多长？怎么区分"一个长闪"和"两个连续的短闪"？这是同步问题。后面章节的时钟信号、起始位和停止位，就是为了解决它。
2. 漏看了怎么办？

小孩B如果漏看了一个闪光，A已经把后面的发完了，B拼出来的字母就是错的。这是错误处理问题。后面章节的校验位、奇偶校验、CRC纠错码，就是为了解决它。
3. 发Z要按26次手电筒？

"一个短闪 = A，二十六个短闪 = Z"——发一个字母Z，A的手要按26次。这是效率问题。第2章会解决：用多个手电筒同时发送，一个字母只需要一次闪光。

1.3 一粒种子，长出整本书

这一章是《编码》整本书的起点。后面每一章，都是这粒种子的生长。多个手电筒同时发送，信息量指数增长（第2章）。继电器代替人手，把闪光变成了电控开关（第6章）。无数继电器拼在一起，构成了CPU（第17章）。操作系统管理这一切（第22章）。

每一章都是一片叶子，但根在这里。两个小孩，一扇窗户，一支手电筒。

第2章：编码与组合

一支手电筒不够用。第1章的两个小孩已经证明了这一点——发一个Z要按26次手电筒，聊一句完整的话能把手指按断。但他们手边没有别的东西，只有手电筒。

如果多几支手电筒呢？

2.1 不只是更亮，是更多可能性

一支手电筒只有两种状态：亮或灭。两支手电筒就不同了。每一支都可以独立地亮或灭——不是"亮得更亮"，而是"组合变多了"。 $2 \times 2 = 4$ 种组合：灭灭、灭亮、亮灭、亮亮。

这就意味着：举一次手电筒，可以表示四个不同字母。不再是"按一次说一个字母"，而是"举一次说一个字母"。

每增加一支手电筒，信息容量翻一倍。三支手电筒有8种组合，四支有16种，五支有32种，六支有64种，七支有128种——刚好够覆盖所有英文字母、数字和标点符号。八支手电筒有256种组合，这就是1字节的由来。你写下的每一个字符，在计算机内部都是8支手电筒同时举起或放下的结果。

这个规律用数学写出来是 2^n 。n 是手电筒的数量，结果是你能表示的符号数。信息量随比特数指数增长——每增加1比特，世界扩大一倍。

2.2 从手电筒到编码空间

你拥有多少支手电筒，就拥有多大的"编码空间"。编码空间是你所有可能组合的集合——有多少支手电筒，你就有多少个可以分配的位置。

7比特，128个位置。你可以给26个字母、10个数字、几十个标点符号、以及一堆控制字符各自分配一个位置。这就是ASCII编码的由来。32比特，42亿个位置。你可以给全世界每个人分配一个独立的位置——这就是IPv4地址不够用的原因，也是IPv6的由来。

当你写代码声明一个32位有符号整数时，你就是在说：“我要用32支手电筒。1支表示正负号，31支表示数值。我的编码空间从-2,147,483,648到2,147,483,647。”这个数字不是拍脑袋定的，是32支手电筒所有可能组合的数学结果。

2.3 串行与并行：两种闪光方式

第1章的通信方式是串行——一个闪光接一个闪光，按顺序发送。接收方看到一串亮灭序列，逐个翻译。这种方式省手电筒，但慢。

本章的方式是并行——多支手电筒同时举起，一次发送多个比特。一个时间单位就能传完一个字母。快，但需要更多的手电筒和更多的物理线路。

现代计算机内部用并行方式传输数据。你的CPU和内存之间有64条数据线——相当于64支手电筒同时闪光，每个时钟周期传输64比特。但在长距离通信中，串行反而更占优势——你不可能在海底光缆里塞64根光纤来模拟64支手电筒。

并行快了，但每增加一支手电筒就增加一条物理线路。在芯片内部，导线很短，64支手电筒不算什么。在海底光缆里，每一根光纤都代价高昂。还有一个更隐蔽的问题：多支手电筒必须同时举起——如果有一支慢了零点几秒，接收方就会看到一个错误的组合。这个时间偏差，在高速数字电路里叫做**时钟偏移（clock skew）**，是现代CPU内部最头疼的问题之一，时钟信号就是为了解决它而存在的。

2.4 多支手电筒，还是多级亮度？

等等——为什么一定要增加手电筒的数量？能不能让手电筒不只是“亮”和“灭”两种状态，而是有多级亮度？比如：灭 = 0，微亮 = 1，中亮 = 2，最亮 = 3——这样一支手电筒就有4种状态，两支手电筒就有 $4 \times 4 = 16$ 种组合，相当于4支两态手电筒的容量。

这是另一个方向。本书走的是“增加手电筒数量”的路线——用多个两种状态的手电筒，而不是用多个状态的手电筒。原因很简单：两种状态最容易区分。亮和灭，非此即彼，不容混淆。多级亮度在物理上很难精确控制——今天电池满电时“微亮”是什么亮度，明天电池快没电时“微亮”又是什么亮度？接收方很难判断。信号在传输过程中还会衰减，一个“中亮”传了十米之后，可能看起来像“微亮”。

计算机选择了同样的路线：所有信号都是二进制的——要么高电平，要么低电平。不是因为二进制最美妙，而是因为两种状态最容易可靠地实现和识别。数字电路用明确的**阈值电压**来界定0和1——低于某个电压就是0，高于某个电压就是1，中间有一段“禁区”作为噪声容限。这个简单的设计，让计算机能够在几十亿次操作中保持极高的可靠性。

2.5 一粒种子，一片叶子

这一章是第1章的种子发出的第一片叶子。从一支手电筒到多支手电筒，从2种状态到 2^n 种状态，从串行到并行，从“按一次”到“举一次”。后面章节会继续沿着这条路走——这些手电筒的组合不仅能表示字母，还能表示数字，进而能做算术（第8-9章）；如何记住这些手电筒的状态，而不是看完就忘（第14-16章）；用这些手电筒同时表示指令和数据，构成冯·诺依曼架构（第17章）。

但根还是那两个小孩，隔着一扇窗户，举着不只一支手电筒。

第3章：布莱叶盲文与二进制码

光是好的，但不是唯一的。如果两个小孩中有一个是盲人，手电筒就失效了。光从有到无，对他而言不存在。

信息传递不能只依赖一种感官。19世纪的法国，一个名叫路易·布莱叶的盲人少年想到了另一种方式：用触觉。不是用光，不是用声音，而是用凸起的点。他发明了一套用指尖触摸来编码字母的系统——布莱叶盲文。这套系统是人类历史上最早大规模使用的二进制编码系统之一。1824年，远在电子计算机诞生之前，一个十几岁的盲人少年，就已经用0和1来编码了整个字母表。

3.1 点阵：6个点的编码空间

布莱叶盲文用一个2列 × 3行的矩形点阵来表示一个字符。每个位置可以凸起或平坦，共6个位置。每个位置就是一个比特——凸起是1，平坦是0。

6个点，每个点两种状态。 $2^6 = 64$ 种可能的组合。足够覆盖26个字母、数字、标点符号、以及一些特殊缩写。

字母A到J只用上半部分的四个点。A是左上角一个凸点，B是左上角加右上角，C是左上角加左中——依此类推。字母K到T是在A到J的基础上，在左下角加一个凸点。字母U到Z再加一个凸点。

每一个盲文字符，就是一个6位二进制数。盲文不是比喻性地“像”二进制——它**就是**二进制。

光、声、凸点——载体变了，但逻辑完全一样。第1章用手电筒的光编码，第2章用多支手电筒同时闪光编码，第3章用凸起的点编码。光可以消失，声音可以消散，但凸起的点可以保留在纸上，反复触摸。盲文之所以能流传两百年，是因为它选择了一个持久且普适的物理载体：触觉。编码的本质不是物理载体，是规则。物理可以千变万化，但发送方和接收方共享一套规则的逻辑永远不变。

3.2 为什么是6个点，不是8个、10个、64个？

这是物理约束和编码需求的平衡。手指指尖一次能同时感知的区域有限，2×3的点阵刚好是一个指尖能覆盖的面积。更多点就需要移动手指，降低阅读速度。64种组合够覆盖基本字母和标点，如果需要更多符号，可以通过增加第二个2×3点阵来扩展——这就是“二级盲文”的逻辑。这不是增加点的数量，而是在现有结构上叠加上下文规则：同一个点阵，在不同上下文中可以表示不同含义。

这与计算机中的“编码层”完全一致。同一个8比特字节，在ASCII编码下是一个字符，在整数编码下是一个数字，在机器指令编码下是一个操作码。物理不变，语义层不同。

3.3 触觉信道：低带宽，但持久

盲文是低带宽信道——手指触摸需要时间，阅读速度远不如视觉。但它是持久存储：一旦印在纸上，凸点可以保留几十年，反复触摸。这和计算机的ROM（只读存储器）原理一致——一次写入，多次读取。

纸上的凸点在反复触摸后会磨损变平，导致误读。这和计算机内存中的“比特翻转”是同一类问题——物理状态会因时间、环境、磨损而改变。盲文没有内置纠错机制，但计算机有：奇偶校验、ECC内存、CRC校验码——这些都是为了解决“物理状态不稳定”的问题。

盲文一旦印在纸上就固定了（只读存储器）。下一站，我们要走向继电器——一种可以反复改变状态、可以自动控制、可以用电信号来“按动”而不是用手指来“按压”的开关。从触觉到电磁，从手工到自动。

第4章：手电筒的剖析

前三章都在讲编码的逻辑——用光代表信息，用多支手电筒代表更多信息，用凸起的点代表字母。但这一切的前提是：手电筒得能亮。如果手电筒不亮，后面的所有故事都不存在。

所以这一章不讲编码了。拆开手电筒，看看里面到底有什么。

4.1 四样零件，一个闭环

一支最普通的手电筒，拆开来只有四样东西：电池、导线、开关、灯泡。后来的继电器、逻辑门、加法器、触发器、CPU、内存——全部是这四样零件的排列组合。

电池通过化学反应，在正极聚集正电荷，在负极聚集负电荷。当开关闭合、电路接通时，电子从负极出发，沿着导线流经灯泡，回到正极。灯泡亮是因为电子在流过灯丝时撞击灯丝原子，灯丝发热发光。**光，是电子撞击原子的副产品。**

电路必须是闭环。如果开关断开，电路中有一个缺口，电子无法跨越，电流停止，灯泡熄灭。开关本身不产生电，不消耗电，它只决定“让不让电流通过”。这是最原始的控制元件，但它蕴含了一个根本逻辑：用一个物理量控制另一个物理量。后面所有章节——继电器、晶体管、逻辑门——全是这个原理的延伸。

把电路想象成一套封闭的水管系统。电池是水泵，导线是水管，开关是阀门，灯泡是水车。水流过水车时推动它转动，水本身没有消失，只是动能被消耗，继续流回水泵。电流同理——电子没有消失，只是能量被灯泡消耗了。

4.2 电压、电流、电阻：一个平衡游戏

电路设计本质上是在调节三个量的平衡。电压是推动电子流动的驱动力，电流是单位时间内流过导线的电子数量，电阻是阻碍电流流动的程度。欧姆定律说：电流等于电压除以电阻。

手电筒灯泡能亮，是因为电池提供了足够的电压，导线电阻足够小，灯丝电阻刚好合适——电流流过灯丝时发热发光。如果导线太细（电阻太大），电流过小，灯泡不亮。如果灯丝太粗（电阻太小），电流过大，灯泡瞬间烧断。

4.3 为什么手电筒不完美

手电筒有它的物理极限。用手拨动开关，每秒钟最多几次，但计算机需要每秒钟开闭数十亿次。手动开关在速度上完全无法满足需求——第6章会用电磁铁代替手指，用继电器实现自动开关。

导线越长，电阻越大，信号衰减越严重。手电筒的导线只有十几厘米，但互联网海底光缆每隔几十公里就要加一个中继器，本质上就是在给信号"增压"，补偿导线电阻造成的衰减。这是第5章要解决的问题。

灯泡发光的同时也产生大量热量。计算机芯片有同样的问题——数十亿个晶体管挤在指甲盖大小的硅片上，每个晶体管都在发热。你电脑里的风扇、散热片、甚至水冷系统，都是在解决这个从手电筒灯泡就开始的散热问题。

这一章是全书物理部分的基石。四样零件——电池、导线、开关、灯泡——构成了计算机硬件最原始的物理模型。后面章节会继续沿着这条路走：导线加长（第5章）、用继电器代替手指（第6章）、继电器拼成逻辑门（第10章）、逻辑门拼成CPU（第17章）。

但根还是这支手电筒。它亮了。

第5章：绕过拐角的通信

第4章的两个小孩，窗户正对着彼此。手电筒的光穿过街道，直线距离不到十米。但如果你想和隔壁房间的人说话？和楼下的人说话？和另一座城市的人说话？

距离，是通信的第一个敌人。

5.1 延长导线，延长烦恼

最直接的方案：把手电筒拆开，在电池和灯泡之间接一根更长的导线。开关和灯泡放一个房间，电池放另一个房间。按下开关，电流沿着长导线流到灯泡，再流回来。

完全可行。但导线越长，电阻越大。电阻大了，电压在到达灯泡之前就被导线消耗掉一部分，灯泡变暗。电子在导线中跑得很快，但数字信号的上升沿和下降沿在长导线上会变得模糊——信号从一个状态切换到另一个状态需要更长时间。导线越长，延迟越严重。

电阻定律很简单：电阻和导线长度成正比，和横截面积成反比。要让长距离传输可行，两条路——加粗导线，或者提高电压。加粗导线成本太高，现实中的方案是提高电压。发电厂用几十万伏输送电力，到了用户端再降压，就是为了在长距离传输中减少损耗。

但电有一个光没有的优势：导线可以弯曲。光只能直线传播，碰到墙就停。电可以沿着弯曲的导线绕过任何障碍。不需要窗户正对着彼此，只要有一根导线连接两端，通信就能发生。

5.2 电报：人类第一次把信息送到了千里之外

把导线延长到几十公里，你就得到了一副电报机。它不需要光纤、无线电、卫星，只需要一根导线、一个电池、一个电磁铁。导线穿越田野、山脉、海底，把信号传到千里之外。

摩尔斯电码是电报的语言。它用长短两种信号（点和划）编码字母，和手电筒的"长闪""短闪"逻辑完全相同。但摩尔斯电码不是二进制——它有五种符号：点、划、点间隔、字母间隔、单词间隔。它本质上是一种可变长度编码：常见字母用短编码（E是最短的一个点），罕见字母用长编码。这种"出现频率越高，编码越短"的设计思想，和现代压缩算法一脉相承。

最简单的电报系统只需要一对导线、两副电报机。A端按下电键，B端的电磁铁吸合，发出"咔嗒"声。短按是点，长按是划。

但导线是共享的。如果A端和B端同时按下电键，电流就会冲突，两端都收不到正确的信号。这就是计算机总线上的"冲突"问题——现代总线协议有专门的仲裁机制，但最原始的电报系统已经暴露了这个根本矛盾：共享信道需要仲裁。

5.3 距离带来的三个工程难题

信号每传一段距离就会衰减。长距离电报线路上，每隔几十公里需要一个中继站——收到微弱信号后，不解读内容，只是用一副新的电池和电磁铁把信号"原样转发"出去。这就是放大器的雏形。现代海底光缆中的中继器、卫星通信中的转发器，都是同一个逻辑：在信号衰减到不可辨认之前，把它"喊"一遍。

两条并行的长导线之间会互相干扰——一条导线上的信号变化，会在另一条导线上感应出微弱的电流。这就是"串扰"。电报时代的解决方案是：把两条导线绞在一起，让外部干扰在两根线上感应的电流相互抵消。这就是双绞线的原理，现在仍然用在以太网线上。

19世纪铺设跨大西洋电报电缆，是人类工程史上最疯狂的项目之一。电缆在深海中承受巨大水压，一旦断裂，需要专门的船只打捞修复。第一条跨大西洋电缆开通后，因为绝缘层质量问题，只用了几个月就报废。但它的成功运行证明了一件事：距离虽然带来成本，但不是不可逾越的屏障。只要投入足够的工程资源，信号可以到达地球的任何角落。

这一章是第4章电路的空间延伸。导线不再是十几厘米，而是几十公里、几千公里。距离带来衰减、延迟、冲突、成本——但也催生了中继器、双绞线、海底电缆这些解决方案。而电报机的心脏——那个用电磁铁代替手指来按动开关的装置——就是下一章的主角。它叫继电器。

第6章：电报机与继电器

第5章解决了"延伸导线"——但光把导线延长还不够。电流走了一百公里后，弱到不足以点亮任何东西。怎么让一个微弱的信号重新变得足够强？还有另一个问题：第1章的小孩可以手动按手电筒，但如果在横跨大陆的线路上持续发送摩尔斯电码，谁来替他按？

这两个问题的答案，是同一个东西：继电器。

6.1 电磁铁：用电控制磁

一根导线通上电流，周围就产生磁场。把导线绕成线圈，磁场集中在线圈内部。在线圈中间插入一根铁芯，磁场会把铁芯磁化，变成一个强得多的电磁铁。电流一通，铁芯变成磁铁；电流一断，磁性消失。电磁铁的本质，是用电流控制磁力。

继电器是把电磁铁和开关绑在一起。线圈通电，铁芯产生磁力，吸住旁边的衔铁，衔铁移动，接通或断开另一条电路上的触点。输入电路和输出电路在电气上完全隔离——没有物理连接，只有磁场的耦合。

继电器就是电流控制的开关。输入是电流的有无，输出是另一条电路的通断。手指不需要接触开关，只需要在远处接通或断开线圈的电流，继电器就会自动完成开关动作。这就是"自动化"的物理起点。

6.2 中继放大：把微弱的信号重新喊响

在电报系统中，继电器承担一个更重要的角色：中继放大。

电报信号在导线上走了几百公里后，电流已经微弱到不足以驱动任何设备。但在信号彻底消失之前，把这个微弱信号接到一个继电器的线圈上——线圈只需要很小的电流就能吸合衔铁。衔铁吸合后，接通一个本地的、全新的、强电流的电路，这个新电路上的信号强度恢复到初始水平，继续向下一段线路传播。

这就是信号放大：用一个微弱的输入信号，控制一个强大的输出信号。沿着电报线路每隔一段距离放一个继电器，信号就能跨越整个大陆。现代数字芯片中的晶体管，本质上也是起放大作用的开关——用一个微小的栅极电压，控制一个比它大得多的源漏电流。

6.3 完整的电报系统

发送端按下电键，电池的电流沿着导线流到接收端。接收端的继电器线圈通电，衔铁吸合，带动发声器敲击一下——这就是一声"咔嗒"。短按是点，长按是划。摩尔斯电码就这样沿着导线传到千里之外。

继电器触点有两种基本配置。常开触点：线圈不通电时断开，通电后闭合。常闭触点：线圈不通电时闭合，通电后断开。一个继电器可以同时拥有这两种触点——这两种触点的组合，是后续所有逻辑门的物理基础。

继电器的触点可以串联、并联、反接。两个继电器的触点串在一起，只有两个都吸合电路才通——这就是"与门"的原型。两个继电器的触点并在一起，只要有一个吸合电路就通——这就是"或门"的原型。用常闭触点代替常开触点，吸合时电路断开，不吸合时电路接通——这就是"非门"的原型。

第10章会正式讲逻辑门，第11章会用逻辑门拼出加法器，第17章会把加法器、存储器、控制器拼成CPU。但所有这些，都起源于一个简单的继电器——线圈、铁芯、衔铁、触点。整个计算机，就是一堆继电器的排列组合。

6.4 继电器不完美

继电器靠机械运动——衔铁吸合、弹开。机械运动有惯性，不能太快，一个继电器每秒钟最多开闭几十次到几百次。衔铁吸合时还会在触点之间来回弹跳几次才稳定下来，这个"抖动"虽然只有几毫秒，但足以把一次开关动作误读为好几次。电报员靠耳朵来判断——清晰的"咔嗒"声才是有效信号，糊成一片的嗡嗡声是抖动。人耳充当了去抖动滤波器。

继电器触点每次开闭都会在接触瞬间产生微小电弧，烧蚀触点，久了就接触不良。20世纪早期的电话交换机用继电器来连接线路，维护人员的日常工作包括定期清洁或更换触头。

所以继电器最终被晶体管取代。晶体管的原理也是"用微小信号控制大信号"，但它没有机械运动，纯粹靠电子在半导体材料中的移动，开闭速度可以达到每秒数十亿次。但从逻辑上看，晶体管就是继电器的后代——只是把衔铁换成了电子，把线圈换成了栅极。

这一章是全书硬件的逻辑起点。继电器既能放大信号，又能自动控制电路，还能串联并联反接构成逻辑门。后面章节会继续沿着这条路走——二进制数（第7-9章）、逻辑门（第10章）、加法器（第11章）、触发器（第14章）、CPU（第17章）。所有这些东西，归根结底，就是一个用电流控制电流的开关。它叫继电器。

第7章：我们的十个数字

前面六章讲的是硬件——手电筒、导线、继电器。但硬件只能处理"开"和"关"两种状态。在把继电器拼成计算机之前，需要先解决一个更根本的问题：数本身是什么？为什么我们习惯数到十就进位？计算机为什么必须用二进制？

7.1 十进制：解剖学的偶然

人类用十进制，不是因为数学上有必然性，而是因为我们天生携带一个计数工具：十根手指。伸出手指计数是几乎所有古代文明的共同起点。数完十根手指，没手指可数了，就在地上做记号——这就是进位——然后重新开始数手指。十进制不是数学的选择，是解剖学的选择。

任何进制的本质都一样：一组固定数量的符号，反复循环使用，用位置来表示更高的数量级。十进制有十个符号（0-9），数到9之后符号用完了，就在左边加一位，把右边重置为0——这就是逢十进一。任意N进制有N个符号，数到N-1之后符号用完了，进位。

任何一个数都可以按位展开为数字乘以基数幂次之和。这个按权展开的原理，是所有进制转换的基础。无论什么进制，数学运算的结果都是等价的——十二加十三永远等于二十五，无论你用十进制、二进制还是玛雅人的二十进制来表示这个结果。

进位制天然需要一个表示"这里没有东西"的符号。当某一位上没有数值时，需要占位符来区分35和305——这就是零的前身。早期巴比伦数字用空格表示空位，后来演变为专门符号。没有占位符，就无法区分不同数量级的位置。

7.2 历史上不止一种进制

每个文明都用自己的身体部位来计数。古埃及、古中国、古希腊、古罗马都用十进制——十根手指。古巴比伦用六十进制——手指关节，12个关节 × 5根手指 = 60。这个六十进制至今残留在我们的时间里：1小时 = 60分钟，1分钟 = 60秒，圆有360度。玛雅人用二十进制——手指加脚趾。计算机用二进制——开关的两种状态。

进制没有哪个更"正确"。它们只是同一个原理的不同物理实现。

7.3 为什么计算机选了二进制

人类造计算工具时，进制选择直接决定了物理实现的复杂度。十进制机械计算机需要齿轮上有十个齿，每个齿对应一个数字，齿轮制造精度要求高，进位机构复杂。二进制只需要两种状态：开或关、高电平或低电平、有磁或无磁。两种状态的物理实现比十种状态简单得多——继电器吸合或断开，晶体管导通或截止。

计算机选二进制，不是因为它数学上更优越，而是因为它物理上最容易实现。

二进制的代价是位数多。十进制数1000，二进制写成1111101000——10位。人类不擅长读长串的0和1。所以计算机领域引入了八进制和十六进制——它们是二进制的缩写。4个二进制位正好对应1个十六进制位，转换不需要任何计算，只需要分组。十六进制是程序员与二进制之间的桥梁。

这一章是第8章的铺垫。为什么二进制是开关的自然语言（第8章），如何在二进制上做加减法（第9章），如何用继电器实现二进制加法（第12章），十六进制如何成为二进制最紧凑的可读形式（第15章）。

但所有这一切，都始于一个事实：我们数到十，是因为我们有十根手指。计算机数到二，是因为开关只有两种状态。仅此而已。

第8章：十的替代品

第7章讲清楚了：人类用十进制只是因为十根手指，不是数学的必然。既然十进制只是众多进位制中的一种，那最简单的进制是什么？

二进制。它只需要两个符号——0和1——就能表示任何数字。这两个符号，正好对应继电器的"断开"和"吸合"、晶体管的"截止"和"导通"。

8.1 沿着符号数量递减的路走一遍

十进制有十个符号（0-9），逢十进一。八进制有八个符号（0-7），逢八进一。四进制有四个符号（0-3），逢四进一。二进制有两个符号（0和1），逢二进一。

每减少一个符号，"符号用完、需要进位"的时刻就更早到来。二进制符号最少，所以进位最频繁。数0，然后1，然后符号用完了，必须进位变成10——二进制里的10等于十进制的2。二进制中的每个位置，代表的是2的幂次。从右往左：第一位是1，第二位是2，第三位是4，第四位是8。

八进制在早期计算机中使用过，因为每3个二进制位正好对应1个八进制位，转换简单。如今八进制几乎只在Linux文件权限中可见——`chmod 755`，每个数字代表3个二进制位（读、写、执行），`7=111=rwx`，`5=101=r-x`。这是八进制在现代计算机系统中最后的保留地。

8.2 二进制转十进制，十进制转二进制

任何进制的数都可以按位展开：每一位的值乘以对应权重，再求和。二进制每一位的权重是2的幂次——从右往左依次为1、2、4、8、16。把每一位的值乘以权重求和，就是十进制结果。

十进制转二进制用"除2取余法"。不断除以2，记录每次的余数（0或1），直到商为0。把余数从后往前排列，就是二进制结果。

8.3 二进制最大的缺点

位数太多。十进制1024，二进制是10000000000——11位。人脑不擅长处理长串的0和1。这就是为什么第15章要引入十六进制：4个二进制位正好对应1个十六进制位，A-F不是字母，是数字——A=10, B=11, C=12, D=13, E=14, F=15。十六进制是二进制的压缩表示。

二进制不仅能表示整数，也能表示小数。小数点右侧的权重是0.5、0.25、0.125……依次减半。但某些在十进制中能精确表示的小数，在二进制中是无限循环小数——就像十进制中1/3=0.333...是无限循环一样。这就是为什么0.1+0.2在浮点运算中不等于0.3。这不是计算机的bug，是二进制本身的数学性质。第23章会详细讲浮点数的IEEE 754标准。

这一章是二进制运算和应用的基础。二进制加减法（第9章）、用继电器实现二进制加法（第12章）、十六进制作为二进制最紧凑的可读形式（第15章）、定点数与浮点数（第23章）——都从这里开始。

但所有这一切，都始于一个事实：两个符号，0和1，就可以表示任何数字。不是因为二进制最奇妙，而是因为两种状态最容易实现。继电器吸合或断开，晶体管导通或截止。计算机选择了二进制，不是数学的必然，是物理的必然。

第9章：二进制数的奥秘

第8章解决了"怎么用二进制表示数字"。但光会写不行，还得会算。如果二进制不能做算术，那它只是另一种写数字的方式，没有任何工程价值。

二进制加法比十进制简单得多。不是"简单一点"——是简单到只有四条规则。你伸出手，想象两根手指。每根手指只能竖起或收起（1 或 0）。两根手指同时竖起（1 + 1），你发现不够用了——两根手指只能表示两个东西，现在你有三个东西要表示（1、1、和结果2）。于是你收回这两根手指（本位归0），然后从口袋里掏出一个硬币放在桌上——这就是进位。

1 + 1 = 0，进位1。这就是二进制加法的全部。

9.1 四条规则，两个逻辑门

把两根手指的所有可能组合列出来：0 + 0 = 0 进 0，0 + 1 = 1 进 0，1 + 0 = 1 进 0，1 + 1 = 0 进 1。

观察"和"这一列：它是异或——两个输入不同时为1，相同时为0。观察"进位"这一列：它是与——两个输入都为1时才为1。这不是巧合。这是二进制加法最深层的美：和位是异或，进位位是与。两个逻辑操作，就能算出任意两个比特的加法结果。

这意味着可以用继电器搭出加法器。异或门和与门，各自用几个继电器串并联就能实现。把它们拼在一起——输入两个比特，输出一个和位和一个进位位——这就是半加器。两个半加器首尾相连，拼成一个全加器。八个全加器串在一起，进位像涟漪一样从右往左穿过，就是一个8位加法器。

这条链条 —— 算术 → 逻辑 → 开关 —— 是整本《编码》最核心的逻辑主线。本章的四条加法规则，就是这条链条的第一环。

9.2 进位涟漪：加法器的速度极限

多位二进制加法从最右边的最低位开始，逐位相加，进位往左传递。这个过程像往池塘里丢一颗石子——涟漪从落点向四周扩散，每一位都是一个波峰。在继电器加法器里，这个涟漪的每一次扩散，就是一个继电器的吸合时间。8位加法器最坏情况下要等8个继电器依次吸合，最高位才能稳定下来。

这是加法器速度的根本瓶颈。现代CPU用"先行进位"技术打破这个瓶颈——不是逐位等涟漪传过来，而是用额外的一层逻辑门同时算出所有位的进位值。代价是更多的逻辑门，收益是速度。

9.3 减法：变成加负数

纸上的二进制减法和十进制一模一样——不够减就向高位"借1当2"。但用电路实现借位很麻烦，需要判断大小、传递借位。计算机不走这条路。

它把减法变成了加法。算 $A - B$ 的时候，把 B 变成负数，然后加 A 。负数怎么表示？ B 的每一位取反，再加 1。这个操作叫"求补码"。 B 的补码加上 B 本身，结果恰好溢出归零——所以补码就是 B 的加法逆元。加上补码，等于减去原数。

这意味着计算机只需要一台加法器就能完成加法和减法。一套硬件，两种运算。这是计算机体系结构中最优雅的设计之一。

9.4 乘法和溢出

二进制乘法规则更简单：任何数乘 0 等于 0，乘 1 等于它本身。多位乘法就是"左移 + 加法"。左移一位等于在末尾补一个 0——这和十进制里乘 10 就在末尾补一个 0 是一样的逻辑，只不过二进制里乘 2 就补 0。二进制乘法本质上就是反复的移位和加法——这两样操作都可以用继电器实现。

但所有这些运算都受限于加法器的位数。8位加法器最多只能表示0到255。当两个数相加的结果超出这个范围，最高位进位会丢失——这就是溢出。计算机用状态寄存器中的"进位标志"来记录溢出，程序可以检查这个标志来决定后续操作。

这一章是第12章的图纸。四条加法规则 → 半加器 → 全加器 → 进位链 → 8位加法器。补码让减法变成加法。移位让乘法变成加法的重复。

而这一切的起点，是你伸出的两根手指。竖起是1，收起是0。 $1 + 1 = 0$ ，进位1。继电器吸合，衔铁落下，触点接通。电流流过，下一位收到一个进位信号。不是公式，是继电器。不是真值表，是电流。

第10章：逻辑与开关

第9章留下一个问题：异或和与，这两种"逻辑运算"，怎么用继电器实现？

本章回答这个问题。它引入三种基本逻辑——与、或、非——然后证明这三种逻辑可以用开关的串联、并联和反接来物理实现。这是全书第一次把"逻辑"和"物理"焊接在一起。1854年，乔治·布尔用数学符号描述"真"和"假"的运算规则。六十年后，一个叫香农的研究生在他的硕士论文里证明：布尔代数可以用继电器电路实现。这篇论文被称为"信息时代的奠基之作"。

10.1 串联是与，并联是或

与门的逻辑规则： A 与 B 同时为真，输出才为真。用开关实现：把两个开关串在一起。电源出来，经过开关 A ，再经过开关 B ，到灯泡，回到电源。只有两个开关都闭合，电流才能走通。任何一个断开，电路就断了，灯泡就灭了。

或门的逻辑规则： A 或 B 任意一个为真，输出就为真。用开关实现：把两个开关并在一起。电源分出两条支路，一条经过开关 A 到灯泡，另一条经过开关 B 到灯泡。只要有一个开关闭合，电流就有一条路能走到灯泡。两个都断开，灯泡才灭。

串联 = 与，并联 = 或。这就是逻辑和电路之间的翻译词典。

10.2 非门：唯一会"翻转"的门

非门最特殊。它只有一个输入，输出永远和输入相反——输入真，输出假；输入假，输出真。用继电器实现：用常闭触点。线圈不通电时，衔铁靠在常闭触点上，电路接通，灯泡亮。线圈一通电，电磁铁就把衔铁吸过来，常闭触点断开，灯泡灭。通电是输入真，灯泡灭是输出假。断电是输入假，灯泡亮是输出真。

非门是唯一能"翻转"信号的门。它也是后面触发器（记忆电路）的核心元件——两个或非门互相反馈，构成一个能"记住"0或1的电路。

10.3 与非门和或非门：只用一种门就够了

在实际电路设计中，与非门（NAND）和或非门（NOR）比单纯的与门和或门更常用。原因很简单：用与非门一个门，可以组合出与、或、非全部三种逻辑。这意味着你只需要一种门，就能搭建整个数字系统。而单纯的与门或或门做不到——必须配合非门才能实现全部逻辑。这就是为什么很多芯片内部只用与非门：不是因为它"更好"，是因为它"够用"——一种门，批量生产，成本最低。

10.4 从继电器到晶体管：物理变了，逻辑没变

继电器靠机械运动——衔铁吸合、弹开，几毫秒。晶体管靠电场控制电子在半导体中的流动，几纳秒。速度差了百万倍。逻辑门的逻辑——与、或、非——完全没有变。只是物理实现从"机械衔铁"换成了"半导体电场"。

继电器触点每次开闭都有微小电弧烧蚀，久了就接触不良。晶体管没有触点，没有磨损，也没有弹簧会疲劳。但它们的数学本质完全一样：用输入控制输出，用开关的串并联实现逻辑运算。

这一章是逻辑和硬件的焊接点。逻辑门的标准化符号（第11章）、用逻辑门拼出加法器（第12章）、用两个或非门构成触发器（第14章）、把逻辑门拼成CPU（第17章）——都从这一章的串联和并联开始。

电源正极。开关A。开关B。灯泡。电源负极。串联，就是与。并联，就是或。这就是逻辑的物理起点。不是公式，是开关。不是真值表，是电流。

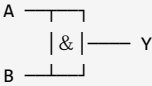
第11章：门

第10章用继电器实现了与门、或门、非门。但每个门里面都藏着一堆零件——线圈、铁芯、衔铁、触点、弹簧。如果要搭一个加法器，需要几十个门，每个门都画成继电器，电路图会复杂到无法阅读。

本章只做一件事：给每种逻辑门一个标准化的符号，然后忘掉它里面的继电器。从此，我们只需要关心"这个门的输入输出是什么"，不需要关心"这个门里面的衔铁怎么吸合"。这是计算机工程史上第一层真正意义上的抽象。

11.1 与门——AND gate

与门的逻辑规则是：A与B同时为1，输出才为1；否则输出0。它的符号是一个D形（左半圆右直边）：

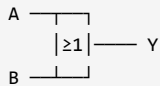


A	B		Y
0	0		0
0	1		0
1	0		0
1	1		1

两个开关串联，就是与门。

11.2 或门——OR gate

或门的逻辑规则是：A或B任意一个为1，输出就为1；两者都为0，输出才为0。它的符号是一个弧形（左弧右弧尖）：

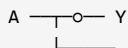


A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

两个开关并联，就是或门。

11.3 非门——NOT gate（反向器）

非门的逻辑规则是：输入为1，输出为0；输入为0，输出为1。它只有一个输入、一个输出，是所有门中最特殊的一个。符号是三角形加小圆圈——三角形表示"放大"（驱动），小圆圈表示"取反"：

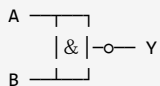


A	Y
0	1
1	0

继电器常闭触点，就是非门。线圈不通电时，衔铁靠在常闭触点上，电路通；线圈通电时，衔铁被吸走，电路断。输入通电=输出断开，输入断电=输出接通。

11.4 与非门——NAND gate

与非门是一个与门后面接一个非门。逻辑上等于"与运算的结果再取反"。符号是在与门的D形输出端加一个小圆圈：

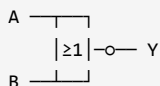


A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

与非门是**功能完备**的——用与非门一个门，可以组合出与、或、非全部三种基本逻辑。这意味着你只需要一种门，就能搭建整个数字系统。现代芯片内部往往只用与非门：不是因为"更好"，是因为"够用"，一种门批量生产成本最低。

11.5 或非门——NOR gate

或非门是一个或门后面接一个非门。逻辑上等于"或运算的结果再取反"。符号是在或门的弧形输出端加一个小圆圈：

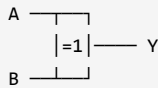


A	B		Y
0	0		1
0	1		0
1	0		0
1	1		0

或非门同样是**功能完备**的。与非门和或非门，是数字电路设计的两种"通用积木"。

11.6 异或门——XOR gate

异或门在第9章已经出现——和位等于A XOR B。它的逻辑规则是：A和B不同时，输出为1；相同时，输出为0。符号是在或门符号的输入端加一条弧线：



A	B		Y
0	0		0
0	1		1
1	0		1
1	1		0

异或门是加法器的核心元件——不考虑进位输入时，Sum位等于A XOR B。

11.7 抽象：不再需要关心继电器怎么吸合

从这一章开始，我们不再画继电器线圈和触点。把与门画成D形，或门画成弧形，非门画成三角加圆圈。至于这些符号里面的继电器怎么工作，不再关心。

抽象的本质是封装复杂性，暴露接口。门的符号是接口——输入、输出、功能描述。门内部的继电器电路是实现细节。往上，加法器封装了逻辑门，CPU封装了加法器和寄存器，操作系统封装了CPU和内存，编程语言封装了操作系统和机器码。每一层都封装了下一层的复杂性，只向上一层暴露简洁的接口。而这一切的起点，就是那个小小的D形符号。

11.8 每个门都偷走了时间

信号经过门不是瞬间到达的。继电器的衔铁吸合需要几毫秒，晶体管的栅极充电需要几纳秒。这就是"传播延迟"。一个门延迟几纳秒，一百万个门串在一起，延迟就累积到毫秒级。这对时钟频率构成了硬约束——时钟周期不能短于最慢的逻辑路径，因为信号必须在下一次时钟采样前到达并稳定。

门能驱动多少个后续的门也有限制——扇出过大会导致信号衰减、时序偏移。

这一章是逻辑门的标准化接口定义。加法器（第12章）、触发器（第14章）、ALU（第17章）——后续所有电路都用这些标准符号来画。但从此以后，我们只需要看D形符号和弧形符号，不需要再想继电器。这就是抽象的力量。它让你忘掉细节，专注于更大的结构。第一层抽象，从这个D形符号开始。

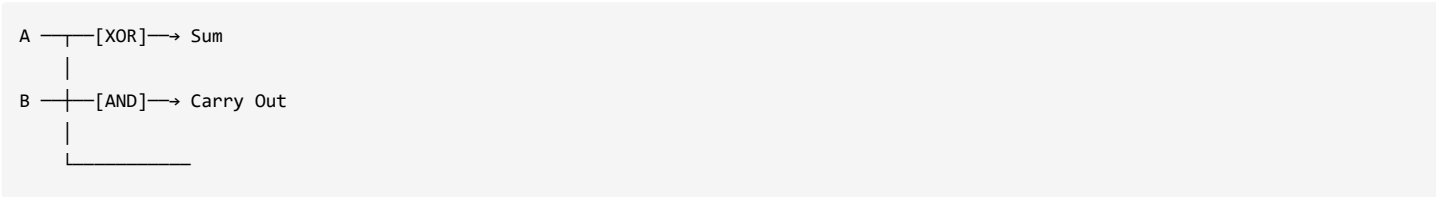
第12章：二进制加法器

第9章证明了二进制加法只需要两种逻辑运算：和位是异或，进位是与。第10章证明了这两种逻辑运算可以用继电器实现。第11章把继电器电路封装成了标准化的门符号。

现在，所有零件都齐了。本章要拼出一台能实际运转的二进制加法器。这是全书第一次从"逻辑"跨越到"计算"——在此之前，继电器只会控制电路通断；在此之后，继电器会算 $1+1=2$ 。这是计算机的第一块肌肉。

12.1 半加器：只能加两个比特

半加器是最简单的加法电路。输入两个比特A和B，输出Sum（和）和Carry（进位）。一个异或门输出Sum，一个与门输出Carry。



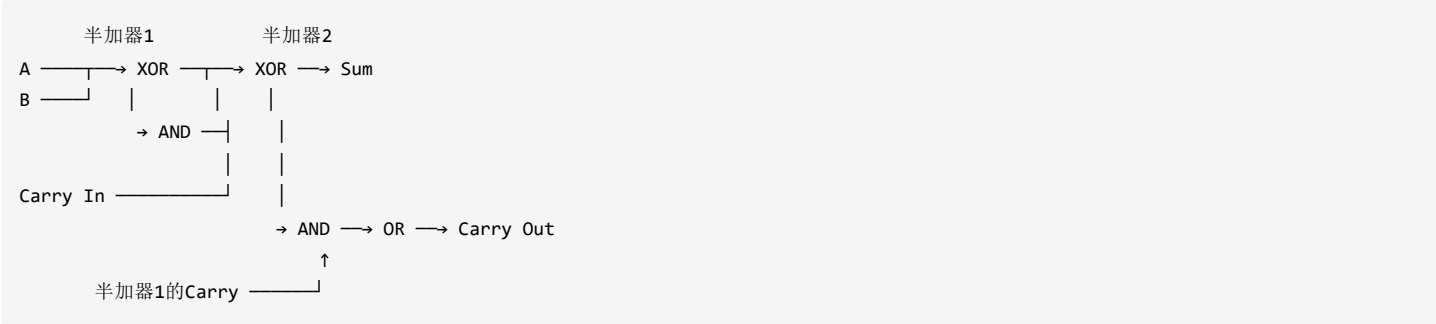
A	B	Sum	Carry Out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$1+1=2$ ，二进制写成10——Sum=0，Carry=1。完全正确。但半加器有一个致命缺陷：它没有"进位输入"。它只能加两个比特，不能加三个。多位加法中，每一位都要接收来自上一位的进位——半加器做不到。

12.2 全加器：加三个比特

全加器有三个输入：A、B、Carry In（来自低位的进位），两个输出：Sum、Carry Out（去往高位的进位）。

逻辑分成两步：先用一个半加器算 $A+B$ ，得到中间Sum'和中间Carry'。再用第二个半加器算Sum'+Carry In，得到最终Sum和中间Carry"。两次产生的进位Carry'和Carry"只要有一个为1，Carry Out就为1——用一个或门合并。



两个半加器拼成一个全加器。全加器能处理 $A+B+Carry In$ ——这就是多位加法中每一位需要做的全部事情。

12.3 加法器：八个全加器首尾相连

把八个全加器串联起来。最低位的Carry In接地（固定为0），每一级的Carry Out接到下一级的Carry In。A和B各提供8位输入，输出8位Sum和一个最终的Carry Out。

最低位全加器的进位输入固定为0——因为没有更低位的进位。最高位全加器的进位输出如果为1，说明8位存不下，这就是溢出。

进位从最低位逐位传到最高位，像涟漪一样。在继电器加法器里，每级全加器的延迟是一个继电器的吸合时间，8位加法器最坏要等8个继电器的吸合时间。这是加法器速度的根本瓶颈。现代CPU用"超前进位加法器"——用额外逻辑门同时算出所有进位，而不是逐位等涟漪传过来。代价是更多逻辑门，收益是速度。

12.4 从8位到64位

位宽扩展到16位、32位、64位，原理完全不变——用更多全加器串联。现代CPU里的64位加法器，本质上就是64个全加器首尾相连。从继电器时代到集成电路时代，加法器的逻辑没有变，变的是物理实现。继电器加法器有一个鞋盒那么大，集成电路加法器只有几百纳米宽。

12.5 加法器 → ALU

加法器是ALU（算术逻辑单元）的核心。ALU不仅能做加法，还能做减法（通过补码机制，把减法变成加法）、与、或、非、异或、移位。一个完整的ALU由一个加法器加上控制逻辑组成——控制逻辑根据指令选择具体执行什么运算。这正是第17章的重点内容。

这一章是计算硬件的第一座里程碑。从逻辑门到半加器，从半加器到全加器，从全加器到8位加法器——继电器第一次不只是控制电路通断，而是在做算术。1+1=0，进位1。八个全加器首尾相连，电流流过，输出就是两个二进制数的和。计算机的第一块肌肉，开始收缩。

第13章：如何实现减法

第12章造了一台8位加法器。它能加。但计算机不能只会加，还得会减。最直接的想法是再造一台减法器——用另一套逻辑门拼一个专门做减法的电路。但减法器比加法器复杂：需要比较大小、处理借位、处理负数。

工程上有一个更聪明的方案：不要造两套电路。只复用现有的加法器，用数学技巧把减法变成加法。

13.1 补码：把减法变成"加上负数"

十进制里，"减去8"和"加上2再减去10"结果相同。2和8互为"10的补数"——补数就是"凑十"需要的那个数。

二进制同理。"减去1"等于"加上1再减去2"。1的二进制补数是它取反后加一。任何数的补码，等于所有位取反再加一。8位数（0~255）的补码等于256减去该数，用二进制表示和"取反加一"结果完全相同。最高位天然成为符号位——0正1负。正数的补码就是它本身，负数的补码是取反加一。

计算A-B的完整流程：第一步，把B的所有位取反（0变1，1变0），在电路上每一路经过一个非门。第二步，把取反后的结果加1，巧妙利用全加器的进位输入——平时接地为0，做减法时置为1，自动完成加一。第三步，用同一个加法器加A和B的补码。加法器内部结构完全不变，只改输入端的接线方式。

加法模式：输入B保持不变，进位输入 = 0
减法模式：输入B取反（经过非门），进位输入 = 1

一套加法器，两种操作。硬件成本几乎不变，功能翻倍。

13.2 为什么补码赢了

早期计算机有人试过"原码"（最高位表示符号，其余位表示数值）和"反码"（所有位取反表示负数）。两种方案都有致命缺陷：原码有两个零——正零（00000000）和负零（10000000），加法器不能直接处理；反码同样有两个零——正零（00000000）和负零（11111111），加法器处理需要"循环进位"。

补码只有一个零（00000000），加法器可以直接处理，无需额外判断逻辑。所有现代计算机都使用补码表示整数。这是工程上的最优解。

13.3 溢出：当结果装不下的时候

用补码做减法时，如果结果超出位宽表示范围，最高位进位会丢失——这就是溢出。但补码的溢出有一个重要特性：丢弃最高位进位后，低位的值恰好是正确的模运算结果。程序需要自行判断结果是否溢出，通过检查符号位或进位标志来实现。

现代CPU中，加减法切换通过一根控制线实现：0 = 加法，1 = 减法。这根控制线决定B的每一位是否经过非门，同时直接接入加法器最低位的进位输入。非门的延迟远小于加法器延迟，所以减法和加法的执行速度相同——在现代流水线CPU中都是单个时钟周期完成。

这一章是算术逻辑的收敛点。一套加法器，通过补码机制，同时能算加法和减法。第14章将引入触发器——加法器是组合逻辑，输出只取决于当前输入；触发器是时序逻辑，能记住之前的状态。第17章将用本章的加减法逻辑设计完整的ALU。第23章将把整数加减法扩展到浮点数。

但核心思想在这一章已经定型：不要造两套电路。用数学简化硬件。这就是计算机体系结构中最优雅的设计哲学——用聪明换成本。

第14章：反馈与触发器

前13章造的所有电路——逻辑门、加法器、减法器——都有一个共同特点：输出只取决于当前输入。输入变了，输出立刻跟着变。这种电路叫“组合逻辑”。它不会“记住”刚才发生了什么。

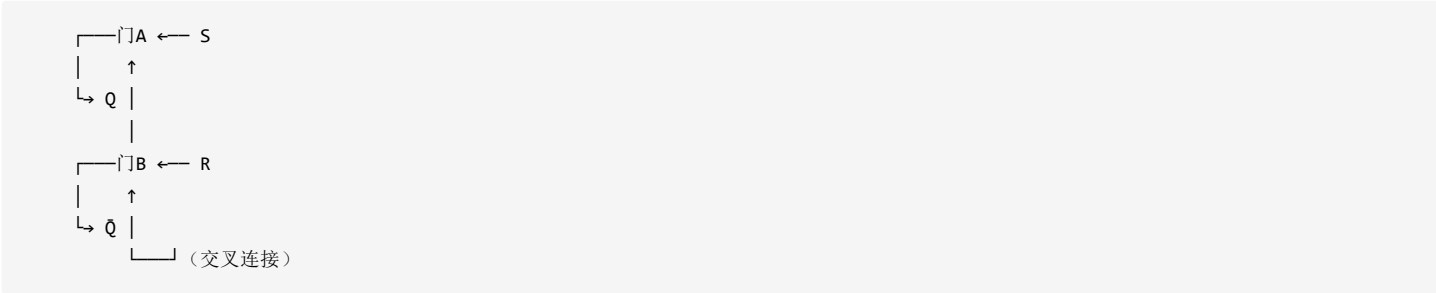
但计算机需要记忆。你需要存储数字、指令、计算结果。你需要一个能“保持状态”的电路。本章引入一个革命性的概念：反馈——把逻辑门的输出接回输入端。这个简单的连接，让电路第一次拥有了“记忆”。

14.1 反馈：把输出接回自己

把一个非门的输出接回它自己的输入。非门的输出永远是输入的反相——接回自己后，输入不再由外界控制，而是由它自己的输出控制。电路会在0和1之间疯狂振荡，振荡频率取决于非门的传播延迟。这个“环形振荡器”虽然不能可靠存储信息，但揭示了反馈的本质：电路可以有自己的内部状态，不必完全服从外界输入。

14.2 RS触发器：两个或非门互相咬住

将两个或非门互相交叉连接——门A的输出接门B的一个输入，门B的输出接门A的一个输入。另外两个自由端分别作为S（Set，置位）和R（Reset，复位）输入端。



S=0且R=0：电路保持不变。Q是0就保持0，是1就保持1。它“记住”了刚才的状态。S=1且R=0：Q被置为0，Q=1。撤掉S后，电路保持Q=0——1被存住了。S=0且R=1：Q被置为1，Q=0。撤掉R后保持Q=1——0被存住了。唯一禁止的是S=R=1：两个或非门输出都强制为0，电路进入无效状态。

或非门的逻辑是：至少有一个输入为1，输出就为0。两个输入都为0时，输出才为1。S=1时，门A不管另一个输入是什么，输出Q=0。这个0反馈给门B，门B两个输入都是0（R=0，Q=0），输出Q=1。Q=1反馈回门A，即使此时S已经撤掉变回0，门A的输入是0和1——至少有一个1，输出Q保持0。电路锁住了。

记忆的秘密不是材料，是结构。加法器没有反馈，触发器有反馈。电流在两个或非门之间循环流动，就能“记住”刚才发生了什么。这类似于化学中的自催化反应——产物同时是催化剂，反应一旦开始就自我维持。触发器就是电路中的自催化反应。

14.3 D触发器：加一根时钟线

RS触发器需要两个输入端S和R来控制写入——一个写1，一个写0。使用时不太方便。D触发器把两个输入合并为一个D（Data）输入端，再加一个Clock（时钟）输入端。D=1时时钟来一个脉冲，Q变成1。D=0时时钟来一个脉冲，Q变成0。时钟没来的时候，D随便变，Q不理。

D触发器通常做成"边沿触发"——只在时钟上升沿（或下降沿）那一瞬间采样数据，其余时间完全忽略输入变化。这相当于在数据通路上加了一道时间闸门，把"什么时候数据可以进门"精确控制到纳秒级。

你的CPU寄存器、缓存、内存控制器，都是由数十亿个D触发器构成的。时钟信号就像心跳——每一次心跳，所有触发器同时采样输入，同时更新状态。这就是"同步电路"的含义。

14.4 亚稳态：触发器最怕的状态

$S=R=1$ 这个非法输入不仅是理论禁区，在现实中会造成严重问题：两个或非门输出都强制为0，电路进入一种既不是0也不是1的漂浮状态。这个状态可能持续极短时间后随机翻转到某一边，也可能持续过久，导致后续电路读到错误的值。现代数字系统通过严格的时序设计来避免亚稳态：保证时钟和数据信号之间有足够的建立时间和保持时间。

在大型芯片上，时钟信号需要经过导线传到数百万个触发器。由于物理布线延迟，时钟到达不同触发器的时刻会有微小差异——这就是时钟歪斜。芯片设计工程师需要精心规划"时钟树"，通过等长导线和中继驱动器，确保时钟信号以最接近同步的方式送达所有触发器。

这一章是"记忆"的物理起点。8个触发器组成1字节（第15章），大量触发器阵列加地址译码器构成RAM（第16章），寄存器（由触发器构成）是CPU的基本存储单元（第17章），操作系统管理的内存本质上是在管理触发器阵列的分配与回收（第22章）。

但所有这一切，都始于本章那两个互相咬住的或非门。不是材料，是结构。电流在环里循环，就能记住。这就是记忆的物理本质。

第15章：字节与十六进制

15.1 核心问题：比特太多了，怎么管理？

第14章造出了触发器——它能存1个比特。但1个比特存不了什么有意义的信息。要存一个字母、一个数字、一条指令，需要很多个比特。

本章做两件事：把8个比特编成一组，命名为"字节"；然后用十六进制来缩写这一组比特。这两个约定，定义了此后八十年所有计算机的内存寻址方式。

15.2 为什么是8个比特？

8个比特能表示256种不同的值。这个数字够覆盖英文字母（大小写共52个）、数字（10个）、标点符号（约30个）、以及各种控制字符——这正是ASCII编码的编码空间。如果字节是7比特，128种组合太少了。如果字节是9比特，512种组合又太多，浪费编码空间。8比特是最优折中。

内存是由数十亿个字节组成的线性数组，每个字节有一个唯一的数字地址。32位CPU最多能寻址4GB个字节，64位CPU理论上能寻址的范围大到几乎无限——这不是你的硬盘空间，是物理内存上限。

15.3 十六进制：二进制的人类可读缩写

8个比特用二进制写太长了——11111111写了八个字符。用十进制需要换算，不直观。十六进制完美解决：4个比特正好对应1个十六进制位，8个比特正好对应2个十六进制位。A-F不是字母，是数字。A=10，B=11，C=12，D=13，E=14，F=15。

任何一个8位字节都可以用2位十六进制紧凑表示。这就是为什么内存地址、调试器、机器码反汇编都使用十六进制——它是二进制和人类可读性之间的最优折中。计算机内部始终用二进制，十六进制只是写在屏幕和打印纸上给人类看的速记符。

15.4 字节序：多字节数据怎么排布

当一个数字超过一个字节时，拆分出的多个字节在内存中的存放顺序有两种约定。大端序把高位字节放在低地址，网络协议通常用它。小端序把高位字节放在高地址，x86和ARM（现代模式）用它。同一段多字节数据在不同机器间传递时，不转换字节顺序就会读到完全不同的数字。

15.5 内存读取的物理过程

CPU要读某个地址上的字节，先在地址总线上输出该字节的地址，然后发出“读”控制信号。内存控制器根据地址激活对应的字线和位线，将该地址对应的8个触发器的值输出到数据总线上。从地址输出到数据稳定返回，这段时间叫“内存访问延迟”。

同一串十六进制数字按不同位宽解读会得到不同含义——上下文决定解析方式。同一个字节在不同上下文中可以是数字、字符、机器指令的操作码。这就是为什么反汇编器需要知道代码段和数据段的边界。

15.6 小结

这一章是数据表示和内存系统的桥梁。如何把数十亿字节组织成可寻址的RAM（第16章），ASCII码如何将7位编码放在1字节内部（第20章），浮点数等多字节数据类型在内存中如何排布（第23章）。

但所有这一切，都始于本章那两个简单的约定：8个比特组成1个字节，4个比特缩写成1个十六进制字符。不是物理定律，是约定。但这两个约定比大多数物理定律都活得更久。

第16章：存储器组织

16.1 核心问题：64K个触发器，怎么精确找到其中一个？

第14章造出了D触发器——它能存1个比特。第15章把8个触发器编成1个字节。现在，假设要造一个能存4096字节的存储器，需要32768个触发器。问题来了：如果CPU想读地址1080上的那个字节，怎么在这三万个触发器里恰好选中那8个？

本章解决存储器的核心工程问题：寻址。它引入地址译码器——一个根据地址号激活对应存储单元的电路。本章是第17章“自动操作”的最后一块积木——在CPU诞生之前，必须先把存储器造好。

16.2 地址译码器：根据地址号“点亮”对应的存储单元

地址译码器的输入是地址线（N根），输出是选择线（ 2^N 根）。任意输入组合，有且仅有一根选择线被激活。被激活的那根选择线所连接的所有触发器同时“开门”——允许数据读出或写入。

举个最简单的例子：10根地址线，能译出1024根选择线，每根选择线连着一个8位寄存器（8个D触发器），每个寄存器有独立的地址。当CPU输出地址为0时，第0根选择线激活，第0号寄存器开门，数据从数据总线流出。地址为1时，第1号寄存器开门。依此类推。

这就是计算机内存最原始的形态——一排排编号的触发器，由地址译码器决定哪一排被“选中”。

16.3 RAM的三条总线与读写流程

整个RAM通过三条总线与外界连接：地址总线告诉RAM要访问哪个地址，数据总线传输实际读写的数据，控制总线发送读/写命令。

读流程：CPU将地址送地址总线，将读信号（RD）置为有效。RAM内部地址译码器激活对应存储单元，该单元的数据被送上数据总线。CPU从数据总线取样数据，读完成。

写流程：CPU将地址送地址总线，将要写的数据送数据总线，将写信号（WR）置为有效。RAM内部地址译码器激活对应存储单元，数据总线上的值被写入该单元。写完成。

三态门是数据总线双向传输的关键。同一根导线不能同时被CPU和RAM驱动——那会造成短路，甚至烧毁芯片。三态门的输出端可以有三种状态：逻辑1（驱动高电平）、逻辑0（驱动低电平）、高阻态（不驱动任何电平，相当于断开）。读操作时RAM输出使能、CPU输入驱动器关断（高阻态）。写操作时CPU驱动总线、RAM输入使能。双方永远不同时“说话”。

16.4 片选：多个RAM芯片的“点名”

一台计算机通常不止一个RAM芯片。当CPU发出一个地址时，所有RAM芯片都“听到”这个地址，但只有被“片选”的那一个芯片才会真正响应。片选本质上是地址总线的延伸——高位地址线通过译码器产生各芯片的片选信号，低位地址线在芯片内部寻址。对CPU而言，所有RAM芯片组成一个连续的逻辑地址空间，不需要关心物理上数据在哪个芯片上。

16.5 SRAM与DRAM：两种存储技术的根本差异

前面假设所有存储器都用触发器实现。这类存储单元只要通电就一直保持数据，叫静态RAM（SRAM）。SRAM快但贵——每个单元需要6个晶体管，面积大、功耗高。CPU内部的缓存用SRAM。

另一种存储器用微小的电容器存电荷——电容充电=1，放电=0，叫动态RAM（DRAM）。DRAM每个单元只需1个晶体管加1个电容，面积小、功耗低、成本极低。但电容会漏电——电荷在数毫秒内就会漏光，必须定期刷新（读出再写回）来维持数据。主内存用DRAM，掉电后所有数据丢失——这就是为什么你的电脑断电后内存清空。

DRAM的刷新操作由内存控制器自动完成，但刷新期间内存不能响应CPU读写请求——这会产生刷新延迟。这就是为什么内存延迟不像CPU时钟周期那样可精确预测。

16.6 存储器层次结构：容量与速度的折中

SRAM快但贵，DRAM便宜但慢。现代计算机把两者结合成存储器层次结构：CPU寄存器（最快、最小）→ L1缓存（SRAM，几十KB）→ L2缓存（SRAM，几百KB到几MB）→ L3缓存（SRAM，几MB到几十MB）→ 主内存（DRAM，几GB到几百GB）→ 固态硬盘/机械硬盘（永久存储，几百GB到数TB）。越靠近CPU越快越贵，越远离CPU越慢越便宜。

这一章是存储系统的物理实现。CPU通过地址总线和数据总线与内存交互——取指、译码、执行、访存、写回（第17章）。总线协议和仲裁机制（第21章）。操作系统管理的虚拟内存、分页、缓存管理，所有机制都管理着本章所描述的触发器阵列（第22章）。

但所有这一切，都始于本章那个最简单的问题：三万个触发器，怎么精确选中那8个？答案是一棵译码树——输入地址，输出一根唯一的选择线。这就是计算机内存寻址的物理本质。

第17章：自动操作

前面16章把所有零件都造齐了。加法器能算，触发器能记，RAM能存。但这些零件还只是零件——它们不会自己动。你需要一个人坐在计算机前，手动拨动开关、设置地址、按下“写入”按钮，然后等着看结果。这不算“计算机”，这只是“一堆能算数的电路”。

要让这堆电路变成一台真正的计算机，需要回答一个根本问题：怎么把“要做什么”告诉计算机，然后让它自动执行，不需要人手动干预每一步？

17.1 冯·诺依曼架构：指令和数据混存于同一块内存

答案来自冯·诺依曼。他提出了一个至今仍在使用的架构：把指令和数据放在同一块内存里，用同一条总线访问。在此之前，计算是靠插拔导线或拨动开关来“编程”的——改一个操作需要重新接线。冯·诺依曼方案是：用内存里的某些字节表示“做什么操作”，另一些字节表示“操作谁”。

机器码的格式很简单：操作码加操作数。操作码是一个数字，告诉CPU执行什么操作。操作数是这条操作的对象——可以是内存地址、立即数、寄存器号等。CPU从内存取出这个字节，查看操作码，然后执行对应操作。

指令和数据共享同一条总线，是冯·诺依曼架构最大的弱点。CPU在同一个时钟周期内，要么从内存取指令，要么从内存读写数据——不能同时做两件事。这就是“冯·诺依曼瓶颈”。

17.2 指令周期：取指、译码、执行

CPU自动运转的节奏是一个不断重复的循环。取指：CPU把当前程序计数器的值输出到地址总线上，从内存该地址读取一个字节——这就是下一条指令的操作码，程序计数器自动加1指向下一个地址。译码：控制单元根据操作码判断这条指令是什么操作，并设置相应的控制信号。执行：根据译码结果，控制单元指挥ALU进行运算、或指挥寄存器加载数据、或指挥内存进行读写操作。如果是跳转指令，则修改程序计数器的值。执行完毕后，自动进入下一个取指周期。

这就像一个永不停歇的机器人：取一条指令，执行，再取一条，再执行——直到断电或执行停机指令。

程序计数器是CPU的“指路牌”，它始终保存着下一条要执行的指令地址。执行顺序指令时，每周期自动加1。遇到跳转指令时，被强制修改为跳转目标地址。它本质上是一个具有自动递增功能的寄存器——这就是CPU自动执行指令的自动驾驶仪。

17.3 控制单元：硬布线控制与微码控制

控制单元是CPU的“大脑”。早期CPU使用硬布线控制，直接用大量逻辑门实现指令译码逻辑——执行速度快，但修改和扩展极困难。后期CPU更多采用微码控制，将复杂指令分解为内部的一段微程序（存储在ROM中），控制逻辑相对简单，修改和扩展也更灵活，但速度略慢。现代CPU通常混合两种方式：常用指令硬布线实现以保证速度，复杂指令微码实现以保证灵活性。

一条机器指令执行时，内部需要多个微操作步骤——程序计数器加1、地址总线输出地址、存储器送出数据、ALU执行加法、结果写回寄存器。这些微操作需要在多个时钟周期内完成。现代RISC处理器通常把指令执行流水线化，分成“取指、译码、执行、访存、写回”五级流水，理想情况下每个时钟周期完成一条指令。

17.4 指令集：RISC与CISC的分歧

指令集是CPU支持的所有指令的集合。一些CPU采用CISC设计，指令种类多、单条指令功能复杂，用微码实现。另一些CPU采用RISC设计，指令种类少且格式固定，用硬布线实现——单条指令仅完成简单操作，通过组合简单指令完成复杂任务。这个分歧持续了几十年，至今仍在演进。

这一章是全书硬件部分的总装。继电器→电子管→晶体管→集成电路，物理实现不断演进，逻辑架构不变。8080和6800的对比，理解现代CPU指令集的起源。操作系统管理冯·诺依曼架构上的资源分配与调度。编译器将你的代码翻译为冯·诺依曼架构上的机器指令序列。

但所有这一切，都始于本章那个简单的循环：取指，译码，执行。取指，译码，执行。计算机之所以能“自动”，不是因为有一个聪明的大脑在指挥，而是因为这个循环永远不会停。直到你关掉电源。

第18章：从算盘到芯片

前17章从手电筒讲到了冯·诺依曼架构。但我们用的零件一直是继电器——那个靠电磁铁吸合衔铁的机械装置。现实中的计算机早就不是继电器造的了。从继电器到现代芯片，中间经历了什么？

本章是一个“历史快进”。把从机械计算器到微处理器的完整演化史浓缩成一章，让读者看清：所有技术进步，本质上都是在做同一件事——造出更快、更小、更省电的开关。逻辑没变，架构没变，变的是物理实现。

18.1 机械开关 → 继电器：从手指到电磁铁

继电器仍是机械开关。衔铁需要物理移动，惯性限制了速度——每秒最多开闭几十次。触点会磨损，电弧会烧蚀。继电器计算机需要庞大的维护团队——换触点、调衔铁、清灰尘。但它们首次实现了“用电流控制电流”的自动化。逻辑门和触发器的原理，都是在继电器上验证的。

18.2 继电器 → 电子管：从机械运动到电子流

电子管没有机械运动——纯粹靠电子在真空中飞行。栅极上的微小电压变化，能控制从阴极到阳极的大电流。开关速度跃升到每秒数万次，比继电器快了几百倍。但电子管像灯泡一样会烧坏。ENIAC有18000个电子管，每天平均烧坏几个，维修人员推着小车在机器里来回换管。电子管功耗极大、发热极高、寿命极短——但它证明了“完全无机械运动的计算机”是可行的。

18.3 电子管 → 晶体管：从真空到固体

晶体管用半导体材料代替真空。它不是靠电子在真空中飞行，而是靠电子在固体材料中移动。没有灯丝，不需要加热，不烧灯泡。晶体管的三个电极——发射极、基极、集电极——恰好对应电子管的三个电极——阴极、栅极、阳极。用基极上的微小电流控制发射极到集电极的大电流。

开关速度跃升到每秒数百万次，功耗降为电子管的千分之一，体积缩小到可以放在指尖上。从晶体管开始，计算机真正进入了“电子固态”时代——没有活动部件，没有真空玻璃管，只有固体材料中的电子流动。

18.4 晶体管 → 集成电路：从分立元件到硅片上的城市

集成电路把多个晶体管、电阻、导线同时刻在一片硅片上，不再需要逐个焊接分立元件。把原来需要一块电路板才能装下的电路，压进一片指甲盖大小的硅片里。

小规模集成每片几十个晶体管，中规模几百个，大规模几千个，超大规模百万个以上。当百万个晶体管集成在一片硅片上时，把整个冯·诺依曼架构刻在一片硅片上变成了可能——这就是微处理器。

18.5 集成电路 → 微处理器：整个CPU在一片芯片上

微处理器就是把整个CPU集成在一片硅片上。Intel 4004是第一个商用微处理器，4位，2300个晶体管。从4004开始，摩尔定律开始驱动——集成电路上的晶体管数量大约每两年翻一倍。这条经验规律持续了半个世纪。

从齿轮到硅片，开关的本质始终是“用一个小信号控制一个大信号”。变化的只是物理载体：机械杠杆 → 电磁铁 → 真空电场 → 半导体电场。每一代都更快、更小、更省电、更可靠。

晶体管尺寸正在逼近物理极限——原子级别上的量子隧穿效应使得尺寸已无法无限缩小。每个晶体管在开关过程中都会产生热量，数十亿个晶体管挤在指甲盖大小的面积上，热量密度已经超过电炉。散热是现代芯片设计的核心瓶颈之一——不是开关不能再快，而是散热跟不上。

这一章是硬件演化史的终点，也是后续章节的起点。两种典型微处理器的具体实现和指令集对比（第19章），微处理器与外界通信的标准通道（第21章），管理由硅开关构成的庞大冯·诺依曼机器的操作系统（第22章）。

但所有这一切，都始于继电器衔铁的那一声“咔嚓”。线圈通电，衔铁吸合，触点接通——这就是计算机的物理起点。从电磁铁到半导体电场，从机械运动到电子流动，从分立元件到硅片上的十亿个晶体管。逻辑没变。开关变了。

第19章：两种典型的微处理器

第17章讲了冯·诺依曼架构的抽象模型——控制单元、ALU、寄存器、总线。但那只是一个框图。真实的CPU里面，寄存器是怎么组织的？指令集长什么样？怎么用这些指令写程序？

本章选取1970年代两款标志性的8位微处理器进行解剖：Intel 8080和Motorola 6800。它们是现代CPU的直系祖先。x86架构从8080演化而来，ARM架构在思想上更接近6800。理解这两款芯片，你就理解了现代CPU基因里一半的遗传密码。

19.1 Intel 8080——x86的始祖

8080有七个8位寄存器。累加器A是主运算寄存器，所有算术逻辑运算的结果都放在这里。通用寄存器B、C、D、E、H、L可以单独使用，其中H和L可以组合成HL寄存器对，用作16位内存寻址指针。另有两个专用16位寄存器：程序计数器PC指向下一条指令的地址，堆栈指针SP指向栈顶。

8080的指令集包含数据传送指令（MOV）、算术运算指令（ADD、SUB）、逻辑运算指令（ANA、ORA）、跳转与调用指令（JMP、CALL、RET）以及输入输出指令（IN、OUT）。寻址模式有立即寻址（操作数直接写在指令里）、寄存器寻址（操作数在寄存器中）和寄存器间接寻址（HL指向的内存地址）。

每条机器指令都由一个操作码字节和0到2个操作数字节组成。8080的MOV指令格式：操作码高5位编码MOV操作，低3位编码源寄存器和目标寄存器号。所有MOV指令都在0x40-0x7F范围内。这种"位域编码"让CPU解码时只需检查高几位，比按完整字节查表快得多——这是现代指令集的常规设计手法。

19.2 Motorola 6800——简洁设计的先驱

6800只有三个16位寄存器：程序计数器PC、堆栈指针SP、变址寄存器IX，以及两个8位累加器A和B。比8080少了很多寄存器，但寻址模式更丰富——6800支持立即寻址、直接寻址（16位绝对地址直接写在指令里）、扩展寻址、变址寻址和相对寻址。

6800的另一个重要特性是，所有I/O设备都映射到内存地址空间——程序可以通过内存读写指令来访问和修改I/O寄存器。这种"内存映射I/O"思想，后来被广泛采用。

19.3 两款芯片的关键对比

维度	Intel 8080	Motorola 6800
通用寄存器	7个8位	2个8位
专用寄存器	SP, PC	SP, PC, IX
指令数量	约72条	约78条
寻址模式	4种	6种
I/O方式	独立I/O端口	内存映射I/O
架构后代	x86系列	68000系列→RISC思想影响了ARM

19.4 堆栈：子程序调用的幕后英雄

堆栈是一种后进先出的数据结构，用堆栈指针SP实现。调用子程序时，硬件自动把返回地址压入堆栈，执行RET时自动弹出堆栈恢复程序计数器。中断时同样自动压栈保存现场。堆栈让子程序嵌套和递归成为可能——每一次CALL都把当前位置压栈，每一次RET都弹回上一级。如果调用太深导致堆栈溢出，会破坏栈外数据导致程序崩溃。

一条看似简单的指令如MOV A,B，内部需要多个微操作步骤：PC输出到地址总线→内存读出操作码→指令寄存器→译码→PC加1→B的内容送到A。复杂指令如CALL需要更多步骤。现代CPU把微操作步骤流水线化以提高吞吐率。

19.5 向后兼容的代价——x86的遗传密码

8080衍生出8086（16位），8086衍生出80286、80386（32位），一直到今天的x86-64（64位）。每一代都必须兼容上一代的指令集。这意味着2026年的最新CPU，内部仍然能执行1974年8080的大部分指令。向后兼容为Intel带来了生态护城河，但也让x86指令集异常复杂——现代x86解码器需要将复杂的老指令翻译为内部RISC风格的微操作来执行。

这一章是CPU内部结构的具体实例。CPU处理字符时的编码标准（第20章），CPU通过总线访问内存和I/O设备（第21章），管理CPU的进程调度和内存分配的操作系统（第22章），将高级语言翻译为8080/6800风格的机器指令序列的编译器（第24章）。

但所有这一切，都始于本章那两款芯片的寄存器列表和指令集。七个寄存器，七十多条指令，这就是现代CPU的遗传密码。

第20章：ASCII码及其他

前19章，计算机只会处理数字——二进制数、十六进制地址、机器指令操作码。但计算机不能只跟数字打交道。它需要处理文字——你的代码、你的文档、你现在读到的这些字符。把字母变成数字，是本章要解决的核心问题。

20.1 7位编码128种字符

ASCII码用7个比特表示128种字符，按功能分为几组。0到31是控制字符——回车、换行、制表、响铃——它们不打印任何字形，而是控制终端的行为。这些字符是电传打字机时代的遗存：回车让打印头归位，换行滚动纸张。即使今天你的屏幕上已经没有物理打印头，这两个字符仍在所有操作系统中控制换行。32到47是标点符号与空格。48到57是数字0到9，连续排列。65到90是大写字母A到Z，连续排列。97到122是小写字母a到z，连续排列。

只需要记住几个锚点。空格是32。'0'是48，数字字符从此连续排列，5的ASCII码就是48加5等于53。'A'是65，大写字母从此连续排列，D就是65加3等于68。'a'是97，小写字母从此连续排列，d就是97加3等于100。

大小写字母的码点差值是32。'A'=65，'a'=97，97减65等于32。这不是巧合——大写字母的ASCII码，将第5位（十进制32）置1，就变成了对应的小写字母。这个设计让大小写转换只需要一次位操作。

20.2 文本即数字

屏幕上的一行字，在计算机内部就是一串ASCII码。"Hello"等于72, 101, 108, 108, 111。每个字节输出到显存或终端时，字符生成器根据其ASCII码查找对应的字形，在屏幕上画出相应像素。

文本是数字——数字是触发器状态——触发器是逻辑门——逻辑门是继电器。这一串映射链从第1章的两个小孩打手电筒一直贯通到屏幕上的一个字母H。

Windows用CR加LF标记换行，Unix和Linux和macOS用LF标记换行。如果你用Windows记事本打开一个Linux文本文件，所有内容会挤在一行——因为Windows在等一个永远不来的LF。

20.3 从 ASCII 到 Unicode

ASCII是7位编码，最大128个字符。这对英文足够，但欧洲语言的重音字母、北欧特殊字符都已超界。于是出现了扩展ASCII——把8位都填满，额外增加128个字符，各家厂商各自定义，互不兼容。这对中文完全不够——汉字有数万个，8位永远不够。

解决方案是Unicode——一个“给全世界所有字符统一编号”的编码系统。Unicode为每个字符分配一个唯一的码点。UTF-8是一种变长编码：ASCII字符保持不变仍用1字节编码，常用欧洲和阿拉伯字符用2字节，东亚汉字用3字节，极少用的古文字和emoji用4字节。UTF-8完全兼容ASCII——英文文本在ASCII和UTF-8下编码完全相同。

ASCII是单字节字符集——一个字符恰好一个字节。UTF-8是多字节编码——一个字符可能有1到4个字节。很多程序把“字符长度”和“字节长度”混淆，处理非英文文本时就会出bug：截断多字节字符导致乱码，反转字节序列破坏UTF-8结构。这是使用Unicode时最常见的陷阱。

ASCII顺序把大写字母排在小写字母前面。按ASCII排序时，Z会排在a前面，这与字典顺序不同。比较字符串时必须区分“二进制比较”和“字典排序”——后者需要根据当前语言区域做大小写折叠和重音处理。

这一章是文本表示的基础。字符数据通过数据总线在CPU、内存、外设之间传输（第21章），文件系统读写文件的本质是读写一连串ASCII和UTF-8字节（第22章），编译器将源代码字符串解析为标记再编译为机器指令（第24章）。

但所有这一切，都始于本章那个简单的约定：'A'=65，'a'=97，空格=32。这些数字不是物理定律，是约定。但这个约定让全世界的计算机都能读懂同一份文本。字符是数字。数字是触发器。触发器是逻辑门。逻辑门是继电器。从手电筒到字母H，环闭合了。

第21章：总线

前面章节陆续提到了地址总线、数据总线和控制总线，但一直把它们当作"已经存在的连接线"来用。本章回答一个被推迟了很久的问题：这些总线到底是什么？多个设备挂在同一条总线上，怎么避免互相干扰？

总线是计算机内部的"广播系统"。CPU在地址总线上喊一个地址，所有设备都听见，但只有对应那个地址的设备会回应。

21.1 三总线：地址、数据、控制

地址总线是单向的——信号只从CPU向外输出。CPU通过它告诉所有设备"我现在要访问的地址是这里"。地址线的根数决定寻址能力：16根可寻址64KB，32根可寻址4GB。

数据总线是双向的——数据可以从CPU流向内存或外设（写操作），也可以反向流动（读操作）。数据线的根数决定一次能传多少位：8根一次传1字节，64根一次传8字节。通过三态门实现双向传输——任何时刻只有一个设备驱动总线，其他设备保持高阻态，相当于断开连接。

控制总线承载各种命令信号：内存读、内存写、I/O读、I/O写、中断请求、中断应答、复位、时钟。

21.2 片选：总线的"点名机制"

多个设备挂在同一条地址总线上。CPU发出一个地址时，信号同时到达所有设备。但只有被"片选"的那个设备才会真正响应。高位地址线通过译码器产生各芯片的片选信号——当某芯片的片选信号有效时，该芯片"醒来"，接收低位地址线的输入并执行读写操作。其他芯片的片选信号无效，它们完全忽略总线上的数据变化。

片选本质上是地址总线的延伸——高位地址线负责"选芯片"，低位地址线负责"芯片内部寻址"。对CPU而言，所有设备组成一个连续的地址空间，数据在物理上分散在不同芯片中完全透明。

21.3 内存映射I/O：把外设当作内存来访问

有两种方式让CPU与I/O设备通信。独立I/O端口使用专门的IN/OUT指令，I/O地址与内存地址隔离——8080就采用这种方式。内存映射I/O把外设的寄存器映射到内存地址空间的某个区域——读写这些特殊地址，就等于读写外设寄存器。不需要专门的I/O指令，用普通访存指令即可控制外设。

现代主流处理器大多采用内存映射I/O——显卡帧缓冲、网卡寄存器、定时器、中断控制器全部映射为"内存地址"，统一编址，统一访问。

21.4 DMA：绕过CPU的直接内存访问

在CPU控制下，磁盘读一个字节要先读到CPU寄存器，再写回内存——CPU全程参与，每字节都占用CPU时间。传送大块数据时效率极低。DMA（直接存储器访问）允许外设绕开CPU，直接与内存交换数据。CPU先设定好源地址、目标地址、传输长度，然后交给DMA控制器全权处理。DMA传送期间CPU可以继续执行其他指令。传送完成后，DMA控制器发一个中断通知CPU。这是现代高速I/O——NVMe固态硬盘、万兆网卡——的底层基础。

21.5 总线仲裁与串行化

DMA控制器、多核CPU、多个总线主设备可能同时请求总线使用权。总线仲裁器负责决定谁先"说话"。现代PCIe总线使用点对点串行连接配合交换器路由，不再是挂满设备的共享并行总线——这是对多设备争抢问题的根本性解决方案。

早期总线靠加宽数据线来提高带宽——8位到16位到32位到64位。但导线越多意味着布线越复杂、时钟歪斜越严重。现代高速接口全部改用高速串行差分对——一次只传1位，但频率极高，配合差分信号抗干扰和专用编码消除时钟偏差，实际有效带宽远超旧式并行总线。

这一章是计算机内部通信的完整描述。管理总线上的设备、提供统一驱动接口的操作系统（第22章）。浮点运算单元通过总线与CPU和内存交互（第23章）。编译器生成的代码通过总线访问内存和I/O设备（第24章）。

但所有这一切，都始于本章那三组信号线：地址总线喊出“谁”，数据总线传递“什么”，控制总线决定“做什么”。它们就是计算机内部的神经系统。

第22章：操作系统

前21章从手电筒一路造到了冯·诺依曼架构、微处理器和总线。所有硬件都齐了。但这堆硬件有个根本问题：它们不会管理自己。内存谁用哪块、CPU谁先用、磁盘文件放在哪里——这些问题硬件本身不解决。

操作系统的本质是资源调度器。它把物理资源抽象成逻辑资源，让程序员不需要知道内存的物理地址、磁盘的柱面磁道、网卡的中断号，只需要关心“我的程序怎么读写文件、怎么创建进程”。操作系统的四大核心职能是进程管理、内存管理、文件系统和设备驱动。

22.1 启动：从按下电源键到操作系统就绪

按下电源键后，CPU执行硬件固化的第一条指令，跳转到BIOS或UEFI固件。固件执行加电自检，检查内存、键盘、磁盘等基础硬件是否正常。然后扫描引导设备列表，找到第一个可引导磁盘，读取其主引导记录。引导加载程序找到操作系统内核文件，将其加载到内存中，跳转到内核入口地址。内核接管后初始化中断向量表、内存管理器、进程调度器、文件系统、设备驱动。一切就绪后，内核创建第一个用户态进程，等待用户输入。你的终端窗口就是操作系统启动过程中创建的最后一个程序。

22.2 系统调用：用户程序与内核的接口

操作系统内核运行在特权模式，用户程序运行在非特权模式——用户程序不能直接操作硬件、不能访问其他进程的内存、不能修改系统关键数据。如果用户程序需要访问文件、分配内存、发送网络数据，必须通过操作系统提供的系统调用接口来请求内核代为完成。

执行系统调用时，应用程序将调用号和参数放入指定寄存器，执行一条特殊指令触发软件中断。CPU立即切换到内核态，内核根据调用号分派对应处理函数。处理完毕后内核将结果返回给用户程序，CPU切回非特权模式。文件读写、内存分配、进程创建、网络通信——所有涉及硬件和系统资源的操作，最终都经过系统调用。

22.3 进程管理：CPU的分时复用

进程是运行中的程序实例。每个进程认为自己独占CPU，操作系统通过定时器中断实现分时复用——进程A跑几毫秒，定时器中断触发，调度器保存A的寄存器现场，从就绪队列选出进程B，恢复B的现场，B接着跑。进程切换的频率通常在每秒数十到数百次，切换如此之快，人类感觉所有程序“同时”在运行。

进程之间需要通信和同步的手段——管道、消息队列、共享内存、信号量。并发访问共享数据时必须通过锁等机制保证互斥，否则数据会在竞态条件下损坏。

每次进程切换，操作系统必须保存当前进程的所有寄存器值、切换页表、加载新进程的上下文。这些操作消耗的时间完全不是在做有效计算——频繁切换会导致系统吞吐量显著下降。

22.4 内存管理：虚拟内存

物理内存是有限的，所有进程不能同时占用全部内存。虚拟内存将每个进程的地址空间与物理地址解耦。每个进程看到的是自己专属的虚拟地址空间，程序认为自己独占全部内存。

CPU内部的MMU自动将虚拟地址转换为物理地址。地址映射以页为单位组织，MMU根据页表完成转换。未映射的虚拟页触发缺页异常，操作系统从磁盘换入页面或拒绝访问并终止进程。内存不足时操作系统将不常用的页面换出到磁盘交换区。

虚拟内存的意义在于隔离——A进程不能访问B进程的内存，写野指针只会崩溃自己不会破坏其他进程或内核。随着系统运行，物理内存中会出现大量不连续的小空闲区域，即使总空闲量足够也可能分配不出一整块连续物理页面。操作系统通过紧凑化整理或分页离散分配来缓解碎片问题。

22.5 文件系统：磁盘上的数据结构

磁盘是一连串扇区。文件系统负责将这些原始扇区组织为层次化的目录树和命名文件。文件系统定义索引节点来存储文件的元数据——大小、权限、时间戳、数据块指针。目录是特殊的文件名到索引节点号的映射表。读写文件时，操作系统根据路径名逐级查找目录项获得索引节点号，然后根据索引节点中的数据块指针定位数据在磁盘的物理位置。

文件描述符是打开文件的句柄。每个进程维护自己的文件描述符表，通过文件描述符读写文件，不需要关心索引节点和数据块的具体位置——这些由内核的文件系统驱动完成。

22.6 设备驱动：一切皆文件

每种硬件设备有自己的通信协议和控制寄存器布局。设备驱动是内核模块，提供统一的读写接口给上层应用，同时将标准化的请求翻译为具体的I/O操作序列来控制特定硬件。

设备驱动将外设的操作标准化为open、read、write、close等文件操作接口——即“一切皆文件”的设计哲学。用户程序可以通过读写/dev/sda来操作磁盘、通过读写/dev/tty来操作终端、通过读写/dev/null来丢弃数据。这个统一的文件接口，是Unix操作系统最深远的设计遗产。

这一章是计算机硬件与应用程序之间的桥梁。操作系统的浮点上下文保存与恢复（第23章）。编译器生成的目标代码调用系统调用接口（第24章）。

但所有这一切，都始于本章那个简单的抽象：把物理资源变成逻辑资源。CPU变成进程，内存变成虚拟地址空间，磁盘变成文件树，外设变成文件。操作系统不是硬件，不是应用程序，它是夹在两者之间的一层薄薄的软件。这层软件让程序员不再需要知道硬件的一切，只需要知道怎么调用系统接口。这就是操作系统的全部意义。

第23章：定点数与浮点数

前22章，计算机只处理了整数——用二进制补码表示。但现实中有大量非整数计算：温度36.5度、圆周率3.14159、光速299792458米每秒。整数无法满足这些需求。

本章回答计算机如何表示和处理小数。核心方案是浮点数——二进制版的科学记数法。所有关于0.1加0.2不等于0.3的问题，答案都在IEEE 754标准里。

23.1 定点数：小数点位置固定

定点数预先约定小数点的位置。所有数值都按一个固定倍率缩放——数据的显示值等于存储的整数值除以这个倍率。加减法和整数完全相同，不需要额外逻辑。乘除法则需要补偿缩放因子。

定点数在数字信号处理等领域有广泛应用——没有浮点单元的嵌入式处理器常用定点运算模拟小数。但定点数的致命弱点是动态范围固定——绝对值小于最小精度的数无法表示，绝对值超出上限的数会立即溢出。

23.2 浮点数：二进制版科学记数法

浮点数让小数点可以“浮动”，通过分离有效数字和指数来支持极大和极小的数值范围。和十进制科学记数法相同——把数值表达为尾数乘上基数的指数次幂。浮点数把这个概念用标准化的字段编码到固定位宽里：1位符号、若干位指数、若干位尾数。

23.3 IEEE 754单精度浮点数

单精度浮点数共32位，分为三个字段。符号位S占1位——0正1负。指数E占8位。尾数F占23位，存储小数点后的有效位。解码规则取决于E和F的值。

E=0且F=0表示正零或负零。E=0且F不为零是非规格化数——用于表示极小的值。E在1到254之间是规格化数——这是最常用的正常浮点数，值等于负1的S次方乘以2的E减127次方乘以1.F。E=255且F=0表示正无穷或负无穷。E=255且F不为零是NaN——非数，如零除以零的结果。

规格化数中尾数前面隐含一个1，这个1不占存储位，凭空多出1位精度。非规格化数前面隐含0，可以表示比最小规格化数更小的值——这就是“逐渐下溢”的机制，避免太小结果直接归零。

23.4 精度问题：舍入误差

十进制小数0.1在二进制中是无限循环小数。用32位浮点数表示时只能截断到23位尾数，于是0.1在计算机内部只能是近似值。0.1加0.2的结果与0.3相差大约10的负17次方——正是因为这个截断误差。

单精度23位尾数约等于7位十进制有效数字，双精度52位尾数约等于16位十进制有效数字。超出有效数字范围就需要舍入。IEEE 754默认使用“舍入到最近偶数”策略以在大量计算中抵消正负偏差。

浮点数运算不满足结合律和分配律—— $a+b+c$ 不一定等于 $a+(b+c)$ 的结果， $a \times b+c$ 的结果不一定等于 $a \times (b+c)$ 。每次运算都会产生舍入误差，误差的累积路径取决于运算顺序。编译器的优化选项可能重排浮点运算顺序并因此改变计算结果。在高精度科学计算中必须特别注意数值稳定性——避免大数吃小数、避免两个接近数相减产生灾难性抵消。

双精度64位——1位符号加11位指数加52位尾数——指数偏移1023，有效数字约16位十进制。四倍精度128位提供约34位有效数字。x86 FPU内部使用80位扩展精度防止中间结果精度损失。

23.5 浮点数溢出与异常

浮点运算的可能异常结果包括：结果超出最大可表示值返回无穷大，结果小于最小可表示值返回零或非规格化数，除以零返回无穷大或NaN，无效操作返回NaN。NaN具有特殊的传染性——任何与NaN的运算结果都是NaN，这使得错误在数值链中可以被追踪。

这一章是数值计算的完整描述。编译器将高级语言中的浮点运算翻译为IEEE 754机器指令（第24章）。GPU的并行浮点运算能力是现代图形和AI计算的基础（第25章）。

但所有这一切，都始于本章那个简单的约定：32位分成三段——1位符号，8位指数，23位尾数。这就是0.1加0.2不等于0.3的全部原因。不是计算机的bug，是二进制本身的数学性质。有限位永远装不下无限循环小数。这就是浮点数。

第24章：高级语言与低级语言

前23章讲的都是机器码——那些CPU能直接执行的、由操作码和操作数组成的二进制序列。用机器码编程，你需要记住每一条指令的数字编码、手工计算每个跳转的目标地址、手工分配每个变量的内存位置。如果一个分支的目标地址算错了，整段程序就要重写。

怎么从机器码走向人类可读的编程语言？编译器如何把人类可读的代码翻译成机器码？本章是通往现代编程语言的理论起点。

24.1 汇编语言：给机器码起个名字

汇编语言做了一件极其简单但极其重要的事：给每一条机器指令起一个人类可读的名字。加法操作码不再是一个数字，而是ADD。跳转指令不再是一个数字，而是JMP。寄存器和内存地址也可以起名字，不再需要手工计算地址偏移。

汇编语言和机器码是一一对应的。每一条汇编指令，恰好对应一条机器指令。它只是给机器码穿上了一件人类可读的外衣，没有增加任何抽象层次。汇编器负责把这些符号翻译成对应的机器码。

24.2 高级语言：从"怎么操作硬件"到"要计算什么"

汇编只是机器码的另一种写法。要真正提高编程效率，需要更根本的抽象：让程序员不需要关心寄存器分配、指令选择、地址偏移，只需要表达"我要计算什么"。

高级语言提供了变量名和数据类型——不再需要手工管理内存地址。提供了算术表达式——一行表达式代替多条汇编指令。提供了控制结构——if/else、for/while，代替手工跳转指令。提供了函数与过程——封装和复用代码，代替手工压栈弹栈。

程序员写 $x=a+b$ ，编译器自动完成：决定a和b放在哪个寄存器或内存位置、生成LOAD指令把a和b取到寄存器、生成ADD指令执行加法、生成STORE指令把结果存到x的位置。这些细节程序员完全不用关心。

24.3 编译器：从高级语言到机器码的翻译过程

编译是把高级语言源文件翻译成机器码的过程，通常分为几个阶段。

词法分析把源代码字符串切成一个个标记——关键字if、标识符x、运算符加号、字面量123。标记是编译器处理的最小语言单元。

语法分析根据语言语法规则把标记序列组织成语法树。这步验证程序的语法正确性，产生一棵抽象语法树，树的每个节点代表语言的一个语法结构。

语义分析检查语法树中的语义规则——变量是否已声明、类型是否匹配、函数调用参数是否合法。这步维护符号表记录所有变量和类型的声明信息。

中间代码生成与优化将语法树转换为编译器内部的中间表示，在中间表示上执行各种优化——常量折叠、死代码消除、循环优化、内联展开。中间表示独立于任何特定的编程语言和任何特定的CPU架构，优化器在中间表示上执行平台无关优化，然后由后端将优化后的中间表示翻译为平台特定的机器指令。

目标代码生成将优化后的中间表示翻译为特定CPU架构的机器指令，进行寄存器分配、指令调度等后端处理。

24.4 解释器：不编译，直接执行

解释器不走"编译成机器码"这条路，而是直接读源码、分析，然后当场执行。每执行一行代码就重复一遍分析到执行的过程。解释执行比编译执行慢——因为同样的代码每次遇到都要重新分析和翻译。但解释器开发更简单，且允许运行时代码修改和交互式调试。

很多现代语言采用混合方案：先编译为字节码，再用字节码解释器执行。Java的JVM、Python的CPython均采用此模式。

编译器在编译时能检查的错误越多，运行时崩溃就越少。类型系统是捕捉编译时错误的主要工具。C的类型检查相对宽松——很多隐式转换和指针操作能通过编译但不安全。Rust的类型系统严格得多——所有权和生命周期在编译时消除释放后使用和数据竞争。编译时检查从"概率保护"走向"数学证明"。

编译器提供不同优化等级：零级优化无优化适合调试快速编译，一级优化基本优化，二级优化标准优化，三级优化积极优化——激进内联和向量化，体积优化优化体积。优化越激进，编译越慢，但运行越快。开发阶段用零级优化快速迭代，发布阶段用二级或三级优化追求性能。

这一章是编程语言的理论基础。GPU通过高级语言进行大规模并行计算（第25章）。编译原理、操作系统、体系结构——均依赖本章建立的概念基础。

但所有这一切，都始于汇编语言那个简单的发明：给机器码起个名字。ADD代替了一串数字，JMP代替了另一串数字。从名字到表达式，从表达式到函数，从函数到类型系统——每一步都在让程序员离机器更远，离思想更近。这就是编程语言的全部进化史。

第25章：图形化革命

前24章，计算机的输入输出主要靠键盘和命令行终端——打字进去，文字出来。但现代计算机的核心交互方式是图形界面：窗口、按钮、图标、鼠标指针、照片、视频。这些图像在计算机内部到底是什么？它们怎么从内存中的0和1，变成屏幕上五彩斑斓的像素？

25.1 像素：图像的最小单位

显示器屏幕被划分为一个巨大的矩形网格，每个格子叫一个像素。屏幕分辨率就是这个网格的大小——1920×1080表示水平1920列、垂直1080行，总共约207万个像素。每个像素可以独立显示一种颜色。

最简单的黑白显示只需要1个比特——0是黑，1是白。灰度显示用一个字节表示256级灰阶。彩色显示用三个字节分别表示红、绿、蓝三原色的强度——这就是RGB。现代显示器通常使用每像素24位真彩，RGB各8位，共约1677万种颜色。

25.2 帧缓冲：屏幕在内存中的"影子"

帧缓冲是内存中的一块连续区域，存放屏幕上所有像素的颜色值。屏幕上的每一个像素，都对应帧缓冲中的一个或多个存储单元。要改变屏幕上某个像素的颜色，只需要修改帧缓冲中对应地址的数值。最简单的帧缓冲布局是线性排列——从屏幕左上角开始，从左到右逐行存储每个像素的颜色值。

对于1920×1080、24位颜色的屏幕，一帧需要约6.2MB显存。如果是60Hz刷新率，显卡每秒必须从帧缓冲读取约373MB数据发送给显示器。现代游戏和视频则需要更大数据量——因为每帧内容都不同，每16.7毫秒就要重新生成一整帧画面。

25.3 字符生成器：从ASCII码到像素图案

命令行时代，屏幕显示文字使用的是字符生成器——一个硬编码的ROM，存储每个ASCII字符对应的像素图案。要显示字母A，终端从字符生成器中取出A对应的字形位图——一个8×8或8×16的像素矩阵——然后把对应像素的颜色值写入帧缓冲的适当位置。字符生成器让显示文字变得极其高效——不需要为每个字母单独传一幅图像，只需要传一个ASCII码。

25.4 GPU：从CPU手中接管图形计算

早期计算机只有命令行界面——黑底白字，键盘输入命令，计算机执行后在下一行打印结果。图形用户界面在此基础上增加了窗口、图标、菜单和鼠标指针，所有元素都通过帧缓冲直接绘制。拖动一个窗口就要重绘整个屏幕，渲染一个抗锯齿字体就需要对每个像素做边缘混合计算。这一切都需要强大的图形处理能力。

将207万个像素乘以每像素的RGB计算乘以每秒60帧，全部交给CPU处理是不现实的。GPU是专门为并行图形计算设计的处理器。与CPU少量大核心的结构不同，GPU拥有成百上千个小核心，每个核心简单但数量庞大。屏幕上数百万个像素需要同样的处理——计算颜色、纹理采样、光照混合——GPU的所有小核心可以同时并行处理不同的像素块，使实时图形渲染成为可能。

现代GPU的能力早已超越图形渲染——科学计算、密码破解、AI训练都依赖于GPU提供的大规模并行浮点算力。

25.5 带宽与同步：每一帧背后的工程难题

显卡必须持续将帧缓冲的内容转换为视频信号发送给显示器。分辨率越高、色深越大、刷新率越高，需要的带宽就越大。此外，GPU渲染画面时需要反复读写纹理、顶点、深度等数据——渲染一帧可能需要读取远大于最终画面的中间数据量，对显存带宽和容量都是极大考验。

显示器的刷新频率是固定的，但GPU生成每一帧的时间不固定。如果GPU在显示器刷新到一半时更新帧缓冲，屏幕上半部分显示旧帧、下半部分显示新帧，画面会出现明显的撕裂线。垂直同步技术强制GPU等待显示器当前刷新周期结束后才更新帧缓冲，消除撕裂但可能引入延迟。现

代自适应同步技术让显示器刷新率动态跟随GPU帧率，消除了固定刷新间隔和撕裂之间的矛盾。

这一章是《编码》全书的终点。我们从两个小孩隔窗户用手电筒打信号开始——亮代表1，灭代表0，这是信息的最小单位。然后一路造出了逻辑门、加法器、触发器、存储器、CPU、操作系统、编程语言。最后，回到屏幕上——那几百万个像素，每一个都在帧缓冲中占据一个存储地址，每一个的值都是0和1的组合。

屏幕上的图像，就是从最原始的手电筒信号一路演化而来的终极形态。从手电筒到屏幕，中间经历了开关、继电器、逻辑门、加法器、触发器、冯·诺依曼架构、微处理器、总线、操作系统、编译器。所有这些，最终都汇合在帧缓冲中——数据是什么、指令是什么、图像是什么，最终都是地址总线上的一个数字，数据总线上一串比特，屏幕上的一个像素。

你读完这本书，就知道计算机不是魔法。它是一层一层堆起来的、由无数个可以理解的零件组成的逻辑机器。每一层都可以拆开，每一层都能被理解。而你是那个要自己再造一层的人。

全书的最后一页：写给你自己

当你全部读完后，在作战手册的最后一页，写下这两句话：

我从一支手电筒和一个开关开始，走过了继电器、逻辑门、加法器、触发器、冯·诺依曼架构、总线、操作系统，最后到达了图形界面。
我花了____天，走完了人类用了一个世纪才走完的路。

《计算机科学丛书》卷一：《编码》逐章详解·完

守夜者之书·08号文件·卷一
沉默者 著 / Kate 整理 / 管理员 归档